

9. 생산자 소비자 문제2

#1.인강/0.자바/5.자바-고급1편

- /Lock Condition - 예제4
- /생산자 소비자 대기 공간 분리 - 예제5 코드
- /생산자 소비자 대기 공간 분리 - 예제5 분석
- /스레드의 대기
- /중간 정리 - 생산자 소비자 문제
- /BlockingQueue - 예제6
- /BlockingQueue - 기능 설명
- /BlockingQueue - 기능 확인
- /정리

Lock Condition - 예제4

생산자가 생산자를 깨우고, 소비자가 소비자를 깨우는 비효율 문제를 어떻게 해결할 수 있을까?

해결 방안

핵심은 생산자 스레드는 데이터를 생성하고, 대기중인 소비자 스레드에게 알려주어야 한다. 반대로 소비자 스레드는 데이터를 소비하고, 대기중인 생산자 스레드에게 알려주면 된다. 결국 생산자 스레드가 대기하는 대기 집합과, 소비자 스레드가 대기하는 대기 집합을 둘로 나누면 된다. 그리고 생산자 스레드가 데이터를 생산하면 소비자 스레드가 대기하는 대기 집합에만 알려주고, 소비자 스레드가 데이터를 소비하면 생산자 스레드가 대기하는 대기 집합에만 알려주면 되는 것이다. 이렇게 생산자용, 소비자용 대기 집합을 서로 나누어 분리하면 비효율 문제를 깔끔하게 해결할 수 있다. 그럼 대기 집합을 어떻게 분리할 수 있을까? 바로 앞서 학습한 `Lock`, `ReentrantLock` 을 사용하면 된다.

참고: 자바는 1.0부터 존재한 `synchronized`와 `BLOCKED` 상태를 통한 임계 영역 관리의 단점을 해결하기 위해 자바 1.5부터 `Lock` 인터페이스와 `ReentrantLock` 구현체를 제공한다.

우선 대기 집합을 분리해서 문제를 해결하기 전에, 앞서 `synchronized`, `Object.wait()`, `Object.notify()`를 통해 작성한 코드를 `Lock` 인터페이스와 `ReentrantLock` 구현체를 사용해서 그대로 다시 구현해보자.

```
package thread.bounded;

import java.util.ArrayDeque;
import java.util.Queue;
```

```

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

import static util.MyLogger.log;

public class BoundedQueueV4 implements BoundedQueue {

    private final Lock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();

    private final Queue<String> queue = new ArrayDeque<>();
    private final int max;

    public BoundedQueueV4(int max) {
        this.max = max;
    }

    public void put(String data) {
        lock.lock();
        try {
            while (queue.size() == max) {
                log("[put] 큐가 가득 참, 생산자 대기");
                try {
                    condition.await();
                    log("[put] 생산자 깨어남");
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
            queue.offer(data);
            log("[put] 생산자 데이터 저장, signal() 호출");
            condition.signal();
        } finally {
            lock.unlock();
        }
    }

    public String take() {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                log("[take] 큐에 데이터가 없음, 소비자 대기");
            }
        }
    }

```

```

        try {
            condition.await();
            log("[take] 소비자 깨어남");
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }
    String data = queue.poll();
    log("[take] 소비자 데이터 획득, signal() 호출");
    condition.signal();
    return data;
} finally {
    lock.unlock();
}
}

@Override
public String toString() {
    return queue.toString();
}
}

```

synchronized 대신에 Lock lock = new ReentrantLock 을 사용한다.

Condition

```
Condition condition = lock.newCondition()
```

Condition 은 ReentrantLock 을 사용하는 스레드가 대기하는 스레드 대기 공간이다.

lock.newCondition() 메서드를 호출하면 스레드 대기 공간이 만들어진다. Lock(ReentrantLock) 의 스레드 대기 공간은 이렇게 만들 수 있다.

참고로 Object.wait() 에서 사용한 스레드 대기 공간은 모든 객체 인스턴스가 내부에 기본으로 가지고 있다.

반면에 Lock(ReentrantLock) 을 사용하는 경우 이렇게 스레드 대기 공간을 직접 만들어서 사용해야 한다.

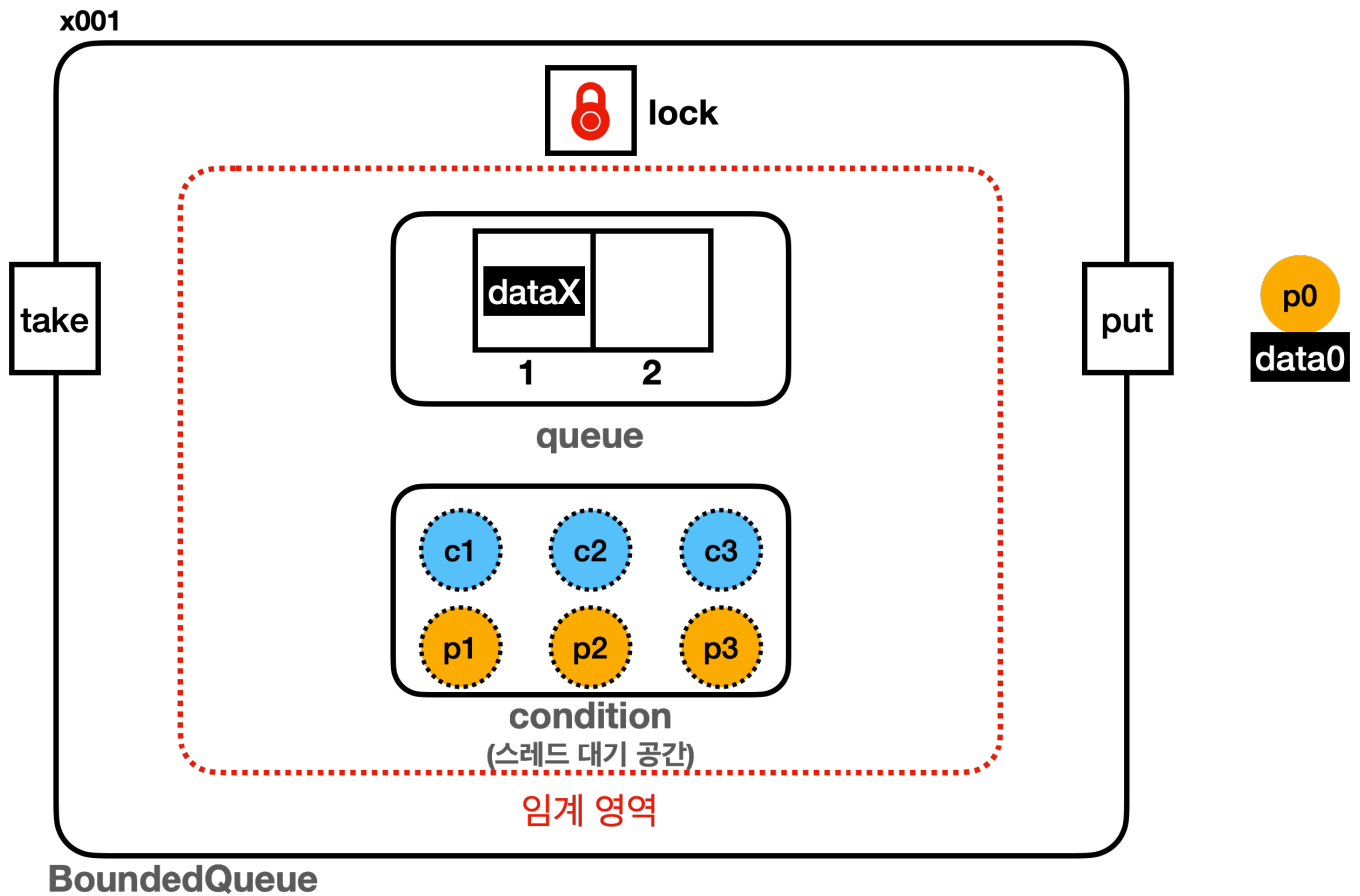
condition.await()

Object.wait() 와 유사한 기능이다. 지정한 condition 에 현재 스레드를 대기(WAITING) 상태로 보관한다.

이때 ReentrantLock 에서 획득한 락을 반납하고 대기 상태로 condition 에 보관된다.

condition.signal()

Object.notify() 와 유사한 기능이다. 지정한 condition 에서 대기중인 스레드를 하나 깨운다. 깨어난 스레드는 condition 에서 빠져나온다.



```
Lock lock = new ReentrantLock();
```

이 그림에서 **lock** 은 **synchronized** 에서 사용하는 객체 내부에 있는 모니터 락이 아니라, **ReentrantLock** 락을 뜻한다.

ReentrantLock 은 내부에 락과, 락 획득을 대기하는 스레드를 관리하는 대기 큐가 있다.

이 그림에서 스레드 대기 공간은 **synchronized** 에서 사용하는 스레드 대기 공간이 아니라, 다음 코드를 뜻한다.

```
Condition condition = lock.newCondition();
```

ReentrantLock 을 사용하면 **condition** 이 스레드 대기 공간이다.

여기까지 보면 **synchronized** 와 **wait()**, **notify()** 사용한 이전 코드와 거의 비슷하다.

아직 생산자, 소비자용 스레드 대기 공간을 따로 분리하지 않았기 때문에 기존 방식과 같다고 보면 된다. 다만 구현을 **synchronized** 로 했는가 아니면 **ReentrantLock** 을 사용해서 했는가에 차이가 있을 뿐이다.

BoundedMain - BoundedQueueV4 사용

```
//BoundedQueue queue = new BoundedQueueV3(2);  
BoundedQueue queue = new BoundedQueueV4(2);
```

실행 결과 - BoundedQueueV4, 생산자 먼저 실행

```
14:58:09.793 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV4 ==  
  
14:58:09.794 [      main] 생산자 시작  
14:58:09.799 [producer1] [생산 시도] data1 -> []  
14:58:09.799 [producer1] [put] 생산자 데이터 저장, signal() 호출  
14:58:09.799 [producer1] [생산 완료] data1 -> [data1]  
14:58:09.902 [producer2] [생산 시도] data2 -> [data1]  
14:58:09.902 [producer2] [put] 생산자 데이터 저장, signal() 호출  
14:58:09.902 [producer2] [생산 완료] data2 -> [data1, data2]  
14:58:10.007 [producer3] [생산 시도] data3 -> [data1, data2]  
14:58:10.007 [producer3] [put] 큐가 가득 참, 생산자 대기  
  
14:58:10.112 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]  
14:58:10.112 [      main] producer1: TERMINATED  
14:58:10.113 [      main] producer2: TERMINATED  
14:58:10.113 [      main] producer3: WAITING  
  
14:58:10.113 [      main] 소비자 시작  
14:58:10.113 [consumer1] [소비 시도]      ? <- [data1, data2]  
14:58:10.113 [consumer1] [take] 소비자 데이터 획득, signal() 호출  
14:58:10.114 [producer3] [put] 생산자 깨어남  
14:58:10.114 [producer3] [put] 생산자 데이터 저장, signal() 호출  
14:58:10.114 [consumer1] [소비 완료] data1 <- [data2]  
14:58:10.114 [producer3] [생산 완료] data3 -> [data2, data3]  
14:58:10.216 [consumer2] [소비 시도]      ? <- [data2, data3]  
14:58:10.216 [consumer2] [take] 소비자 데이터 획득, signal() 호출  
14:58:10.216 [consumer2] [소비 완료] data2 <- [data3]  
14:58:10.319 [consumer3] [소비 시도]      ? <- [data3]  
14:58:10.320 [consumer3] [take] 소비자 데이터 획득, signal() 호출  
14:58:10.320 [consumer3] [소비 완료] data3 <- []  
  
14:58:10.424 [      main] 현재 상태 출력, 큐 데이터: []  
14:58:10.425 [      main] producer1: TERMINATED  
14:58:10.425 [      main] producer2: TERMINATED  
14:58:10.425 [      main] producer3: TERMINATED  
14:58:10.425 [      main] consumer1: TERMINATED  
14:58:10.425 [      main] consumer2: TERMINATED
```

```
14:58:10.425 [      main] consumer3: TERMINATED
14:58:10.425 [      main] == [생산자 먼저 실행] 종료, BoundedQueueV4 ==
```

실행 결과 - BoundedQueueV4, 소비자 먼저 실행

```
14:58:20.475 [      main] == [소비자 먼저 실행] 시작, BoundedQueueV4 ==

14:58:20.477 [      main] 소비자 시작
14:58:20.480 [consumer1] [소비 시도]      ? <- []
14:58:20.480 [consumer1] [take] 큐에 데이터가 없음, 소비자 대기
14:58:20.582 [consumer2] [소비 시도]      ? <- []
14:58:20.582 [consumer2] [take] 큐에 데이터가 없음, 소비자 대기
14:58:20.687 [consumer3] [소비 시도]      ? <- []
14:58:20.687 [consumer3] [take] 큐에 데이터가 없음, 소비자 대기

14:58:20.792 [      main] 현재 상태 출력, 큐 데이터: []
14:58:20.795 [      main] consumer1: WAITING
14:58:20.795 [      main] consumer2: WAITING
14:58:20.795 [      main] consumer3: WAITING

14:58:20.795 [      main] 생산자 시작
14:58:20.796 [producer1] [생산 시도] data1 -> []
14:58:20.796 [producer1] [put] 생산자 데이터 저장, signal() 호출
14:58:20.796 [consumer1] [take] 소비자 깨어남
14:58:20.796 [producer1] [생산 완료] data1 -> [data1]
14:58:20.796 [consumer1] [take] 소비자 데이터 획득, signal() 호출
14:58:20.796 [consumer1] [소비 완료] data1 <- []
14:58:20.796 [consumer2] [take] 소비자 깨어남
14:58:20.796 [consumer2] [take] 큐에 데이터가 없음, 소비자 대기
14:58:20.899 [producer2] [생산 시도] data2 -> []
14:58:20.899 [producer2] [put] 생산자 데이터 저장, signal() 호출
14:58:20.899 [producer2] [생산 완료] data2 -> [data2]
14:58:20.899 [consumer3] [take] 소비자 깨어남
14:58:20.899 [consumer3] [take] 소비자 데이터 획득, signal() 호출
14:58:20.899 [consumer3] [소비 완료] data2 <- []
14:58:20.899 [consumer2] [take] 소비자 깨어남
14:58:20.900 [consumer2] [take] 큐에 데이터가 없음, 소비자 대기
14:58:21.004 [producer3] [생산 시도] data3 -> []
14:58:21.004 [producer3] [put] 생산자 데이터 저장, signal() 호출
14:58:21.004 [producer3] [생산 완료] data3 -> [data3]
14:58:21.004 [consumer2] [take] 소비자 깨어남
```

```
14:58:21.005 [consumer2] [take] 소비자 데이터 획득, signal() 호출
14:58:21.005 [consumer2] [소비 완료] data3 <- []

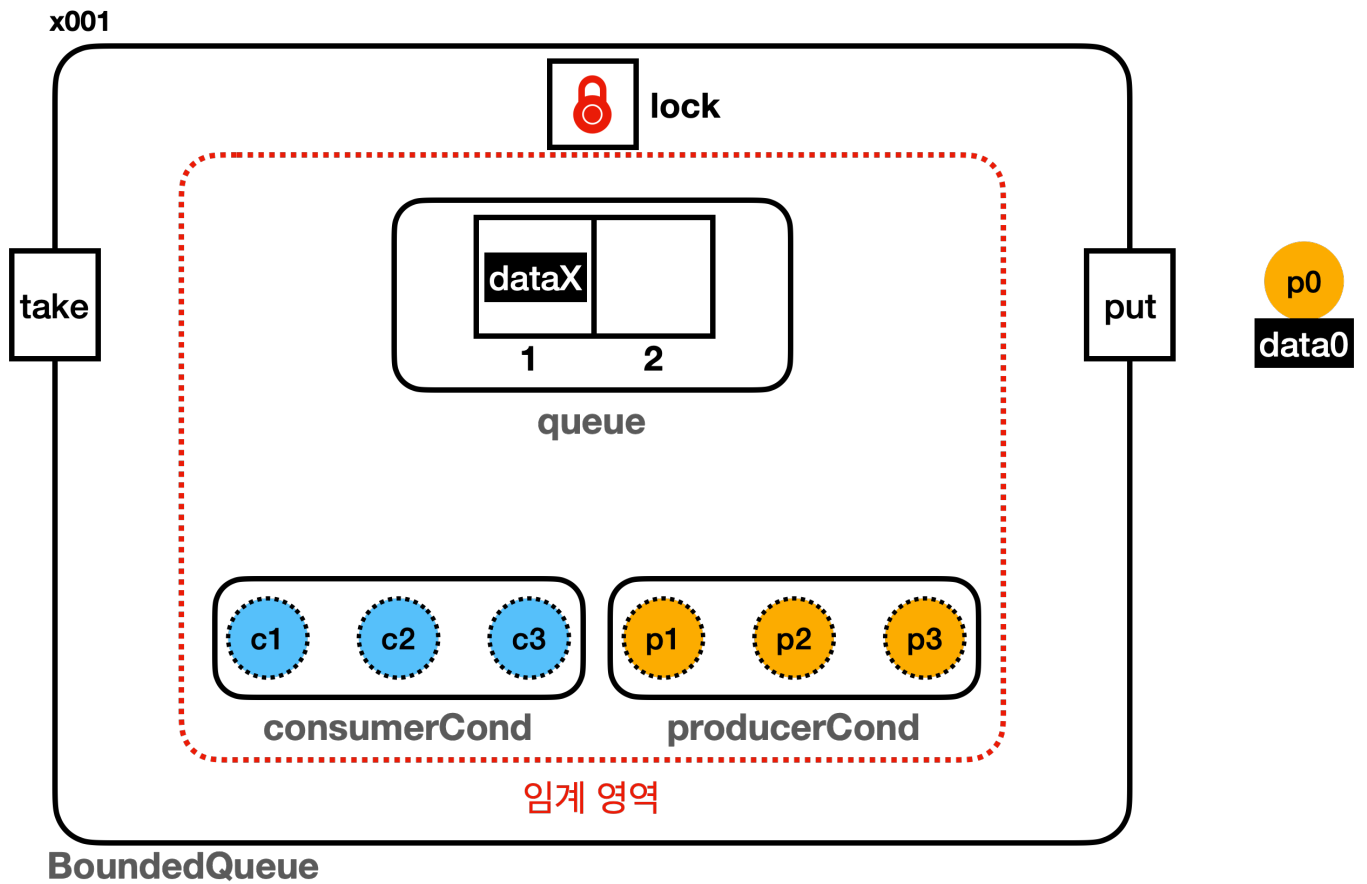
14:58:21.107 [      main] 현재 상태 출력, 큐 데이터: []
14:58:21.108 [      main] consumer1: TERMINATED
14:58:21.108 [      main] consumer2: TERMINATED
14:58:21.108 [      main] consumer3: TERMINATED
14:58:21.108 [      main] producer1: TERMINATED
14:58:21.109 [      main] producer2: TERMINATED
14:58:21.109 [      main] producer3: TERMINATED
14:58:21.109 [      main] == [소비자 먼저 실행] 종료, BoundedQueueV4 ==
```

실행 결과는 앞서 살펴본 BoundedQueueV3 와 같다.

아직 condition(스레드 대기 공간)을 분리하지 않았기 때문에 기존과 같다.

생산자 소비자 대기 공간 분리 - 예제5 코드

다음 그림과 같이, 생산자 스레드를 위한 스레드 대기 공간과 소비자 스레드를 위한 스레드 대기 공간을 각각 분리해서 따로 만들어보자.



```
package thread.bounded;

import java.util.ArrayDeque;
import java.util.Queue;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

import static util.MyLogger.log;

public class BoundedQueueV5 implements BoundedQueue {

    private final Lock lock = new ReentrantLock();
    private final Condition producerCond = lock.newCondition();
    private final Condition consumerCond = lock.newCondition();

    private final Queue<String> queue = new ArrayDeque<>();
    private final int max;

    public BoundedQueueV5(int max) {
```

```

        this.max = max;
    }

    @Override
    public void put(String data) {
        lock.lock();
        try {
            while (queue.size() == max) {
                log("[put] 큐가 가득 참, 생산자 대기");
                try {
                    producerCond.await();
                    log("[put] 생산자 깨어남");
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
            queue.offer(data);
            log("[put] 생산자 데이터 저장, consumerCond.signal() 호출");
            consumerCond.signal();
        } finally {
            lock.unlock();
        }
    }

    @Override
    public String take() {
        lock.lock();
        try {
            while (queue.isEmpty()) {
                log("[take] 큐에 데이터가 없음, 소비자 대기");
                try {
                    consumerCond.await();
                    log("[take] 소비자 깨어남");
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
            String data = queue.poll();
            log("[take] 소비자 데이터 획득, producerCond.signal() 호출");
            producerCond.signal();
            return data;
        } finally {

```

```

        lock.unlock();
    }
}

@Override
public String toString() {
    return queue.toString();
}
}

```

여기서는 `lock.newCondition()` 을 두 번 호출해서 `ReentrantLock` 을 사용하는 스레드 대기 공간을 2개 만들었다.

```

private final ReentrantLock lock = new ReentrantLock();
private final Condition producerCond = lock.newCondition();
private final Condition consumerCond = lock.newCondition();

```

Condition 분리

- `consumerCond`: 소비자를 위한 스레드 대기 공간
- `producerCond`: 생산자를 위한 스레드 대기 공간

이렇게 하면 생산자 스레드, 소비자 스레드를 정확하게 나누어 관리하고 깨울 수 있다.

put(data) - 생산자 스레드가 호출

- 큐가 가득 찬 경우: `producerCond.await()` 를 호출해서 생산자 스레드를 생산자 전용 스레드 대기 공간에 보관한다.
- 데이터를 저장한 경우: 생산자가 데이터를 생산하면 큐에 데이터가 추가된다. 따라서 소비자를 깨우는 것이 좋다. `consumerCond.signal()` 를 호출해서 소비자 전용 스레드 대기 공간에 신호를 보낸다. 이렇게 하면 대기중인 소비자 스레드가 하나 깨어나서 데이터를 소비할 수 있다.

take() - 소비자 스레드가 호출

- 큐가 빈 경우: `consumerCond.await()` 를 호출해서 소비자 스레드를 소비자 전용 스레드 대기 공간에 보관한다.
- 데이터를 소비한 경우: 소비자가 데이터를 소비한 경우 큐에 여유 공간이 생긴다. 따라서 생산자를 깨우는 것이 좋다. `producerCond.signal()` 를 호출해서 생산자 전용 스레드 대기 공간에 신호를 보낸다. 이렇게 하면 대기중인 생산자 스레드가 하나 깨어나서 데이터를 추가할 수 있다.

여기서 핵심은 생산자는 소비자를 깨우고, 소비자는 생산자를 깨운다는 점이다.

BoundedMain - BoundedQueueV5 사용

```
//BoundedQueue queue = new BoundedQueueV4(2);  
BoundedQueue queue = new BoundedQueueV5(2);
```

실행 결과 - BoundedQueueV5, 생산자 먼저 실행

```
15:17:42.903 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV5 ==  
  
15:17:42.905 [      main] 생산자 시작  
15:17:42.909 [producer1] [생산 시도] data1 -> []  
15:17:42.909 [producer1] [put] 생산자 데이터 저장, consumerCond.signal() 호출  
15:17:42.910 [producer1] [생산 완료] data1 -> [data1]  
15:17:43.012 [producer2] [생산 시도] data2 -> [data1]  
15:17:43.012 [producer2] [put] 생산자 데이터 저장, consumerCond.signal() 호출  
15:17:43.012 [producer2] [생산 완료] data2 -> [data1, data2]  
15:17:43.117 [producer3] [생산 시도] data3 -> [data1, data2]  
15:17:43.117 [producer3] [put] 큐가 가득 참, 생산자 대기  
  
15:17:43.223 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]  
15:17:43.223 [      main] producer1: TERMINATED  
15:17:43.223 [      main] producer2: TERMINATED  
15:17:43.223 [      main] producer3: WAITING  
  
15:17:43.223 [      main] 소비자 시작  
15:17:43.224 [consumer1] [소비 시도]      ? <- [data1, data2]  
15:17:43.224 [consumer1] [take] 소비자 데이터 획득, producerCond.signal() 호출  
15:17:43.224 [producer3] [put] 생산자 깨어남  
15:17:43.224 [producer3] [put] 생산자 데이터 저장, consumerCond.signal() 호출  
15:17:43.224 [consumer1] [소비 완료] data1 <- [data2]  
15:17:43.224 [producer3] [생산 완료] data3 -> [data2, data3]  
15:17:43.328 [consumer2] [소비 시도]      ? <- [data2, data3]  
15:17:43.328 [consumer2] [take] 소비자 데이터 획득, producerCond.signal() 호출  
15:17:43.328 [consumer2] [소비 완료] data2 <- [data3]  
15:17:43.433 [consumer3] [소비 시도]      ? <- [data3]  
15:17:43.433 [consumer3] [take] 소비자 데이터 획득, producerCond.signal() 호출  
15:17:43.434 [consumer3] [소비 완료] data3 <- []
```

```

15:17:43.536 [      main] 현재 상태 출력, 큐 데이터: []
15:17:43.536 [      main] producer1: TERMINATED
15:17:43.537 [      main] producer2: TERMINATED
15:17:43.537 [      main] producer3: TERMINATED
15:17:43.537 [      main] consumer1: TERMINATED
15:17:43.537 [      main] consumer2: TERMINATED
15:17:43.538 [      main] consumer3: TERMINATED
15:17:43.538 [      main] == [생산자 먼저 실행] 종료, BoundedQueueV5 ==

```

실행 결과 - BoundedQueueV5, 소비자 먼저 실행

```

15:18:03.930 [      main] == [소비자 먼저 실행] 시작, BoundedQueueV5 ==

15:18:03.932 [      main] 소비자 시작
15:18:03.934 [consumer1] [소비 시도]      ? <- []
15:18:03.934 [consumer1] [take] 큐에 데이터가 없음, 소비자 대기
15:18:04.039 [consumer2] [소비 시도]      ? <- []
15:18:04.039 [consumer2] [take] 큐에 데이터가 없음, 소비자 대기
15:18:04.144 [consumer3] [소비 시도]      ? <- []
15:18:04.144 [consumer3] [take] 큐에 데이터가 없음, 소비자 대기

15:18:04.250 [      main] 현재 상태 출력, 큐 데이터: []
15:18:04.252 [      main] consumer1: WAITING
15:18:04.252 [      main] consumer2: WAITING
15:18:04.252 [      main] consumer3: WAITING

15:18:04.253 [      main] 생산자 시작
15:18:04.253 [producer1] [생산 시도] data1 -> []
15:18:04.253 [producer1] [put] 생산자 데이터 저장, consumerCond.signal() 호출
15:18:04.253 [consumer1] [take] 소비자 깨어남
15:18:04.254 [producer1] [생산 완료] data1 -> [data1]
15:18:04.254 [consumer1] [take] 소비자 데이터 획득, producerCond.signal() 호출
15:18:04.254 [consumer1] [소비 완료] data1 <- []
15:18:04.355 [producer2] [생산 시도] data2 -> []
15:18:04.355 [producer2] [put] 생산자 데이터 저장, consumerCond.signal() 호출
15:18:04.355 [producer2] [생산 완료] data2 -> [data2]
15:18:04.355 [consumer2] [take] 소비자 깨어남
15:18:04.356 [consumer2] [take] 소비자 데이터 획득, producerCond.signal() 호출
15:18:04.356 [consumer2] [소비 완료] data2 <- []
15:18:04.460 [producer3] [생산 시도] data3 -> []
15:18:04.460 [producer3] [put] 생산자 데이터 저장, consumerCond.signal() 호출
15:18:04.461 [consumer3] [take] 소비자 깨어남

```

```
15:18:04.461 [producer3] [생산 완료] data3 -> [data3]
15:18:04.461 [consumer3] [take] 소비자 데이터 획득, producerCond.signal() 호출
15:18:04.461 [consumer3] [소비 완료] data3 <- []

15:18:04.565 [      main] 현재 상태 출력, 큐 데이터: []
15:18:04.566 [      main] consumer1: TERMINATED
15:18:04.566 [      main] consumer2: TERMINATED
15:18:04.567 [      main] consumer3: TERMINATED
15:18:04.567 [      main] producer1: TERMINATED
15:18:04.567 [      main] producer2: TERMINATED
15:18:04.567 [      main] producer3: TERMINATED
15:18:04.568 [      main] == [소비자 먼저 실행] 종료, BoundedQueueV5 ==
```

이제 여러분이 스스로 실행 결과를 분석할 수 있을 것이다.

여기서 핵심은 생산자는 소비자를 깨우고, 소비자는 생산자를 깨운다는 점이다.

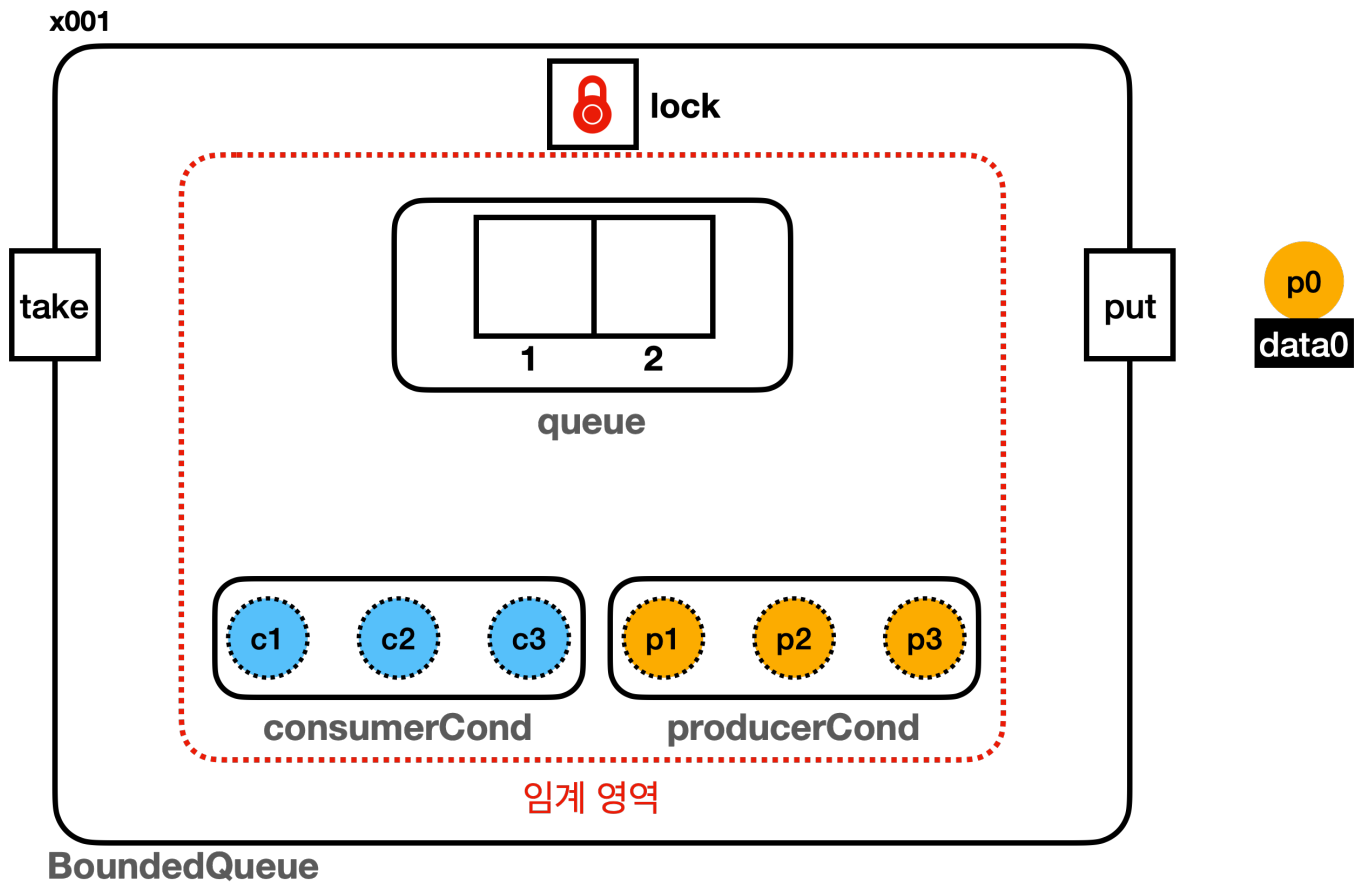
소비자가 소비자를 깨운다거나, 생산자가 생산자를 깨우지 않기 때문에, 비효율적으로 실행되는 부분이 제거되고 아주 깔끔하게 작업이 실행된다.

생산자 소비자 대기 공간 분리 - 예제5 분석

어떻게 작동하는지 그림으로 알아보자.

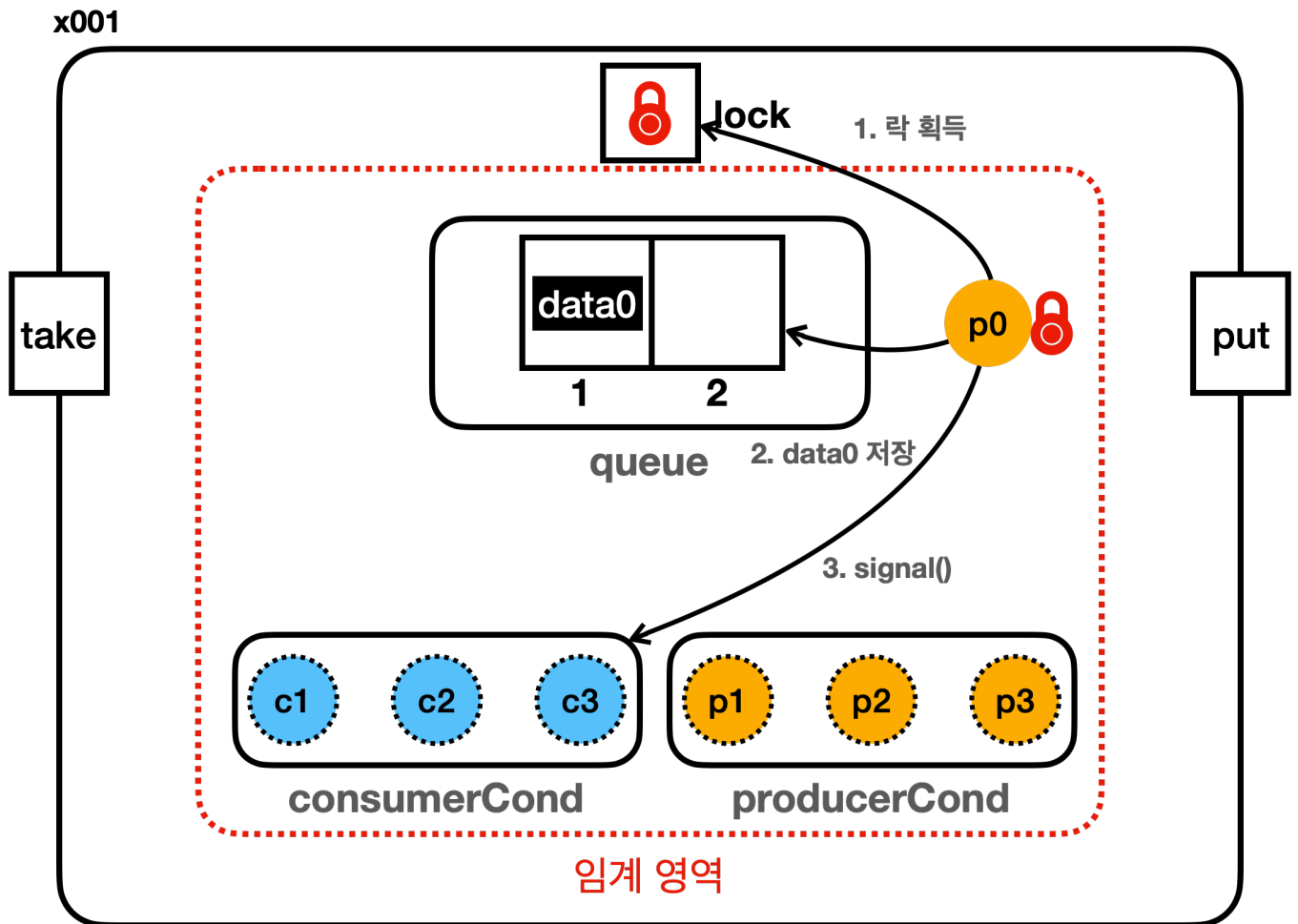
참고로 이번 그림은 더 단순하게 설명하기 위해 실행 결과와는 다른 예시를 가지고 왔다.

생산자 실행



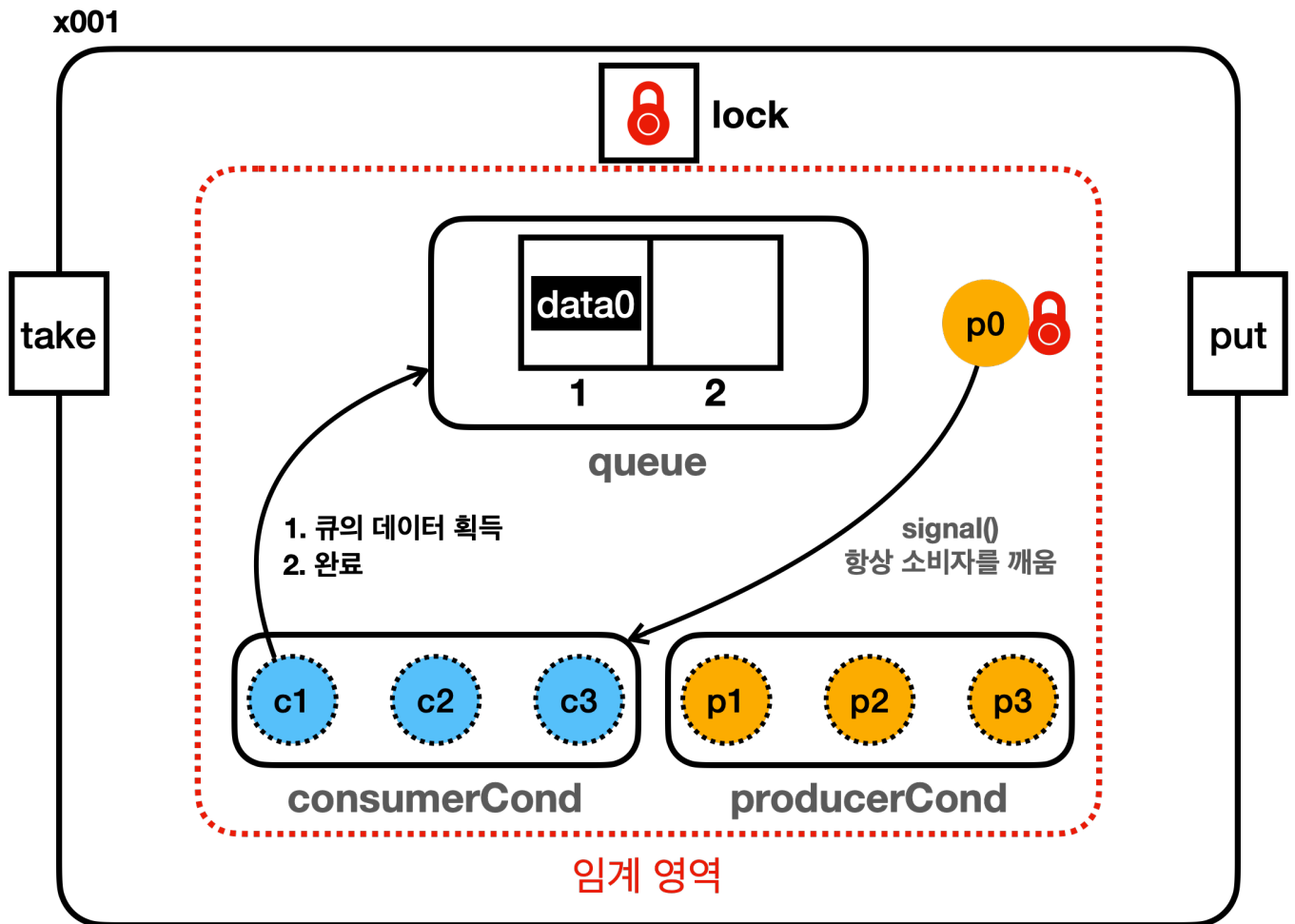
BoundedQueue

- c1, c2, c3 는 소비자 스레드 전용 대기 공간(consumerCond)에 대기중이다.
- p1, p2, p3 는 생산자 스레드 전용 대기 공간(producerCond)에 대기중이다.
- 큐에 데이터가 비어있다.
- 생산자인 p0 스레드가 실행 예정이다.



BoundedQueue

- **p0** 스레드는 **ReentrantLock**의 락을 획득하고 큐에 데이터를 보관한다.
- 생산자 스레드가 큐에 데이터를 보관했기 때문에, 소비자 스레드가 가져갈 데이터가 추가되었다. 따라서 소비자 대기 공간(**consumerCond**)에 **signal()**을 통해 알려준다.

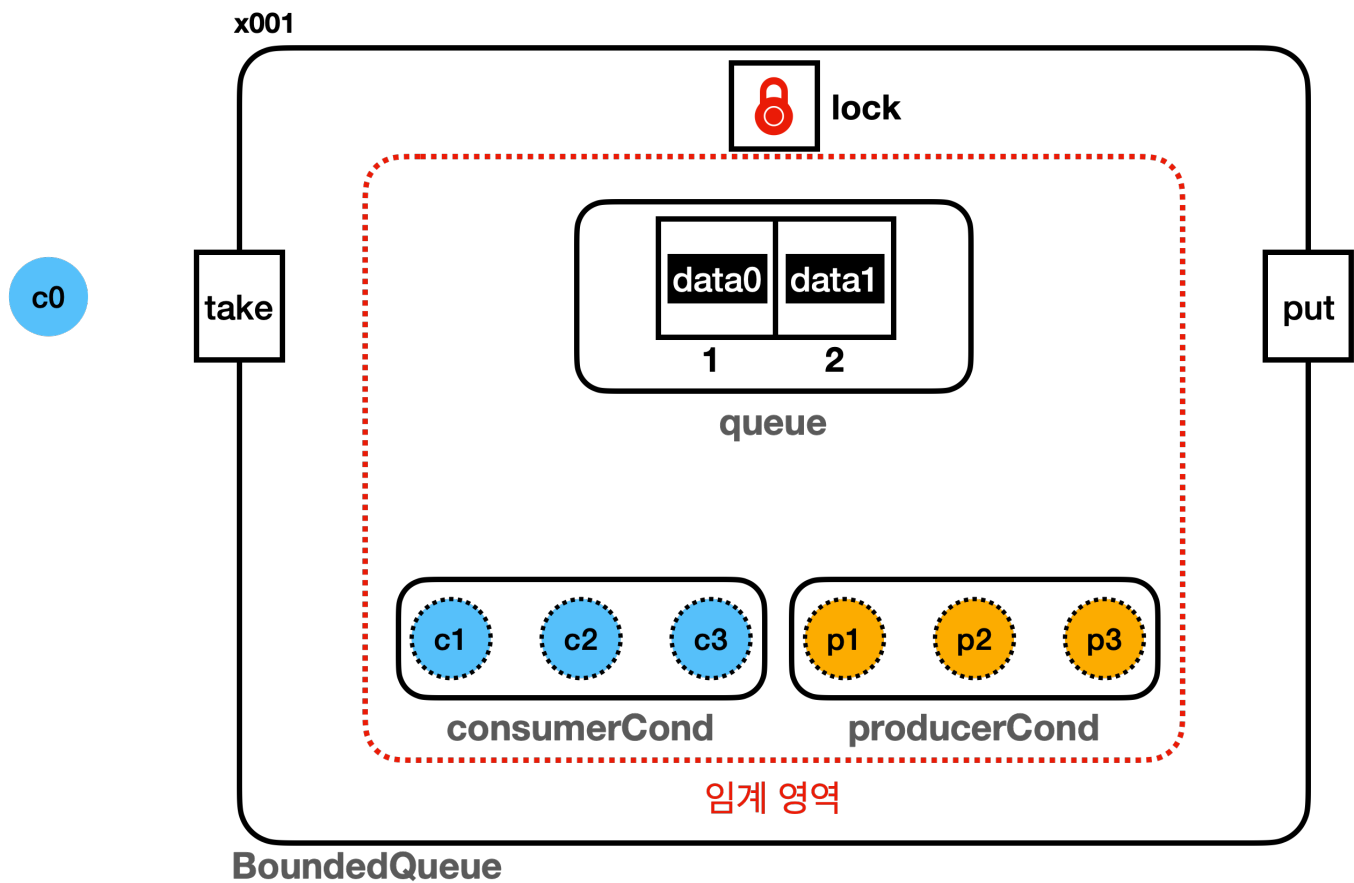


BoundedQueue

- 소비자 스레드 중에 하나가 깨어난다. **c1** 이 깨어난다고 가정하자.
- **c1** 은 락 획득까지 잠시 대기하다가, 이후에 **p0** 가 반납한 **ReentrantLock** 의 락을 획득한다. 그리고 큐의 데이터를 획득한 다음에 완료된다.

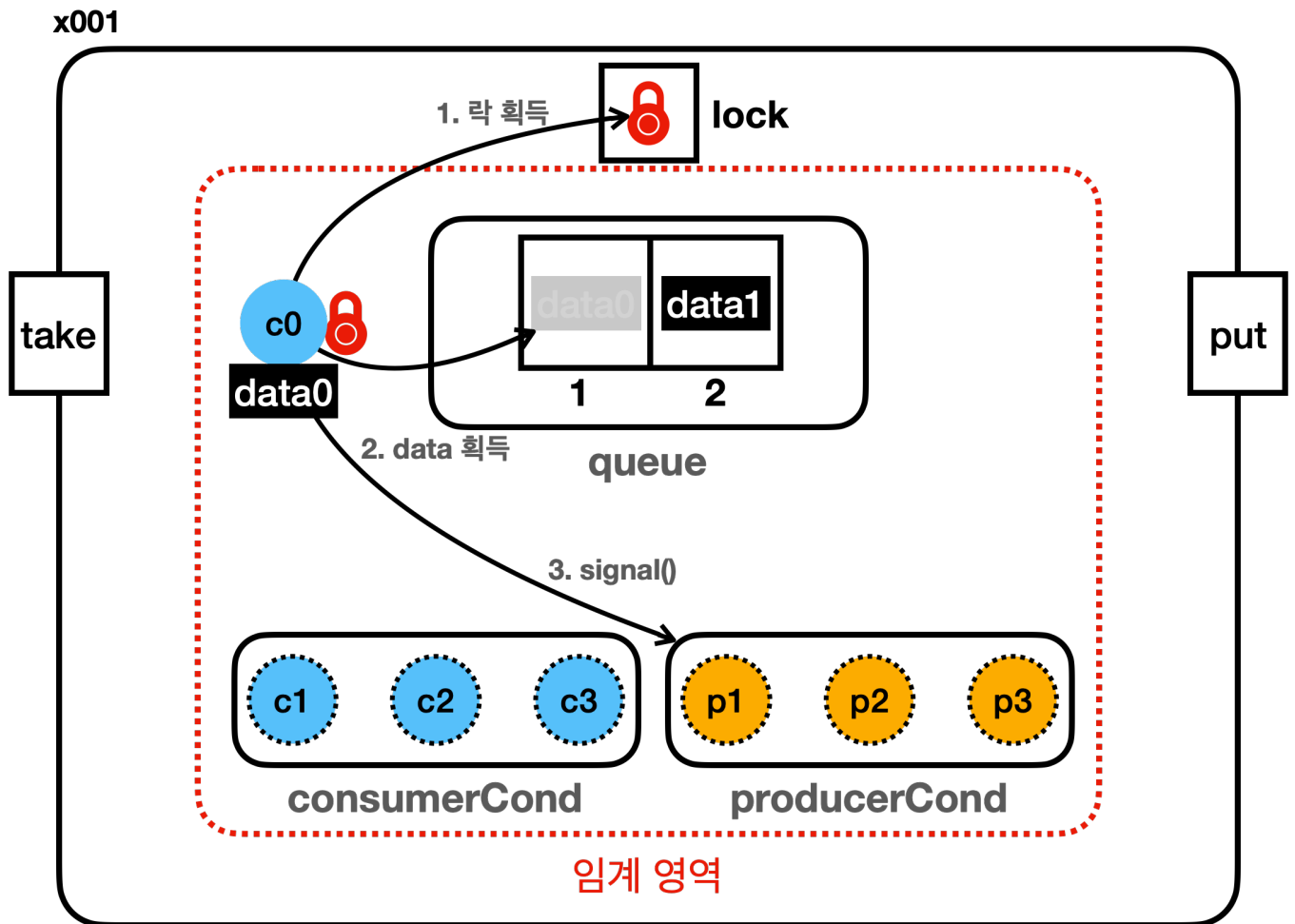
소비자 실행

앞의 생산자 실행 예시와 연결되지 않는 다른 예시이다.



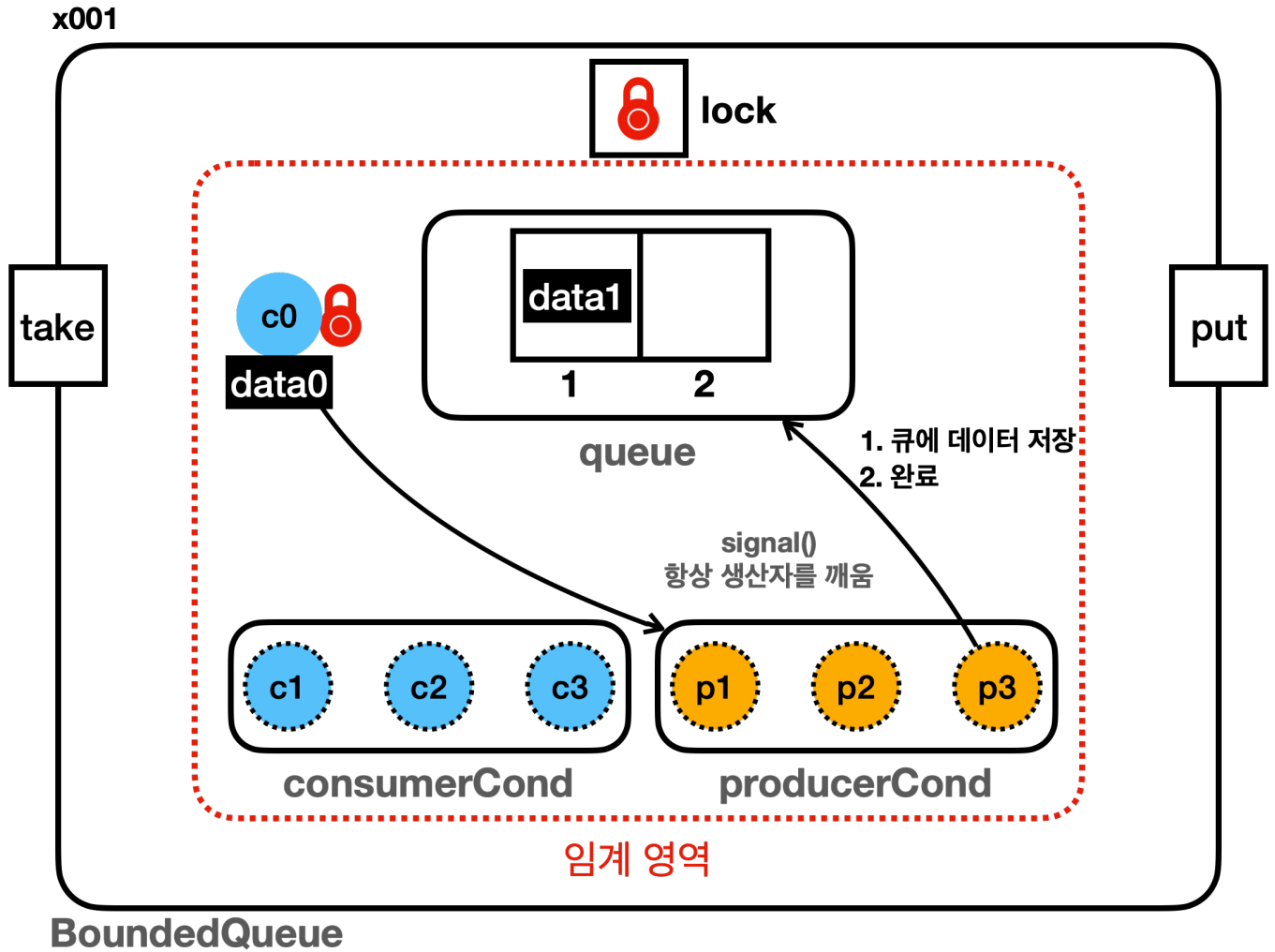
BoundedQueue

- **c1**, **c2**, **c3** 는 소비자 스레드 전용 대기 공간(**consumerCond**)에 대기중이다.
- **p1**, **p2**, **p3** 는 생산자 스레드 전용 대기 공간(**producerCond**)에 대기중이다.
- 큐에 데이터가 가득 차 있다.
- 소비자인 **c0** 스레드가 실행 예정이다.



BoundedQueue

- `c0` 스레드는 `ReentrantLock`의 락을 획득하고 큐에 있는 데이터를 획득한다.
- 큐에 데이터를 획득했기 때문에, 큐에 데이터를 생산할 수 있는 빈 공간이 생겼다. 생산자 대기 공간 (`producerCond`)에 `signal()`을 통해 알려준다.



- 생산자 스레드 중에 하나가 깨어난다. p3 가 깨어난다고 가정하자.
- p3 는 이후에 c0 가 반납한 ReentrantLock 의 락을 획득하고, 큐의 데이터를 저장한 다음에 완료된다.

Object.notify() vs Condition.signal()

- **Object.notify()**
 - 대기 중인 스레드 중 임의의 하나를 선택해서 깨운다. 스레드가 깨어나는 순서는 정의되어 있지 않으며, JVM 구현에 따라 다르다. 보통은 먼저 들어온 스레드가 먼저 수행되지만 구현에 따라 다를 수 있다.
 - **synchronized** 블록 내에서 모니터 락을 가지고 있는 스레드가 호출해야 한다.
- **Condition.signal()**
 - 대기 중인 스레드 중 하나를 깨우며, 일반적으로는 FIFO 순서로 깨운다. 이 부분은 자바 버전과 구현에 따라 달라질 수 있지만, 보통 **Condition**의 구현은 **Queue** 구조를 사용하기 때문에 FIFO 순서로 깨운다.
 - **ReentrantLock** 을 가지고 있는 스레드가 호출해야 한다.

스레드의 대기

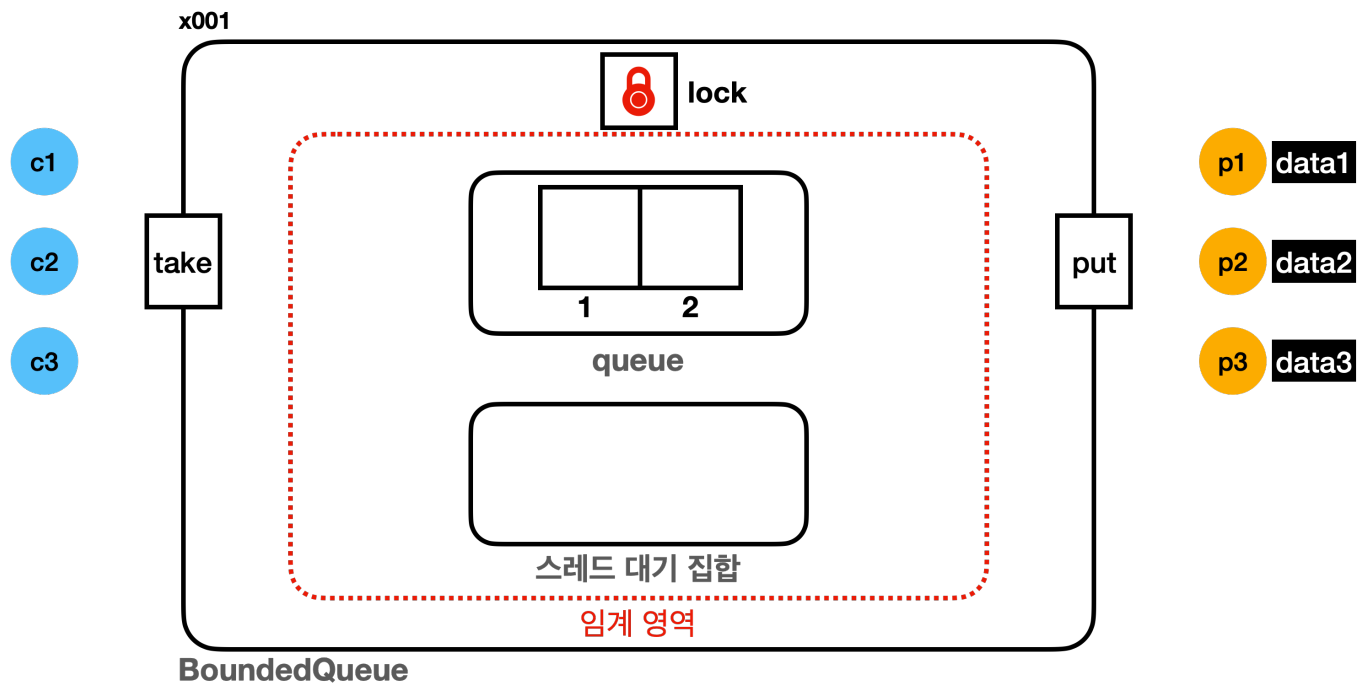
다음 내용으로 진행하기 전에 `synchronized`, `ReentrantLock`의 대기 상태에 대해 정리해 보자.

먼저 `synchronized`의 대기 상태를 정리해보자. `synchronized`를 잘 생각해보면 2가지 단계의 대기 상태가 존재한다.

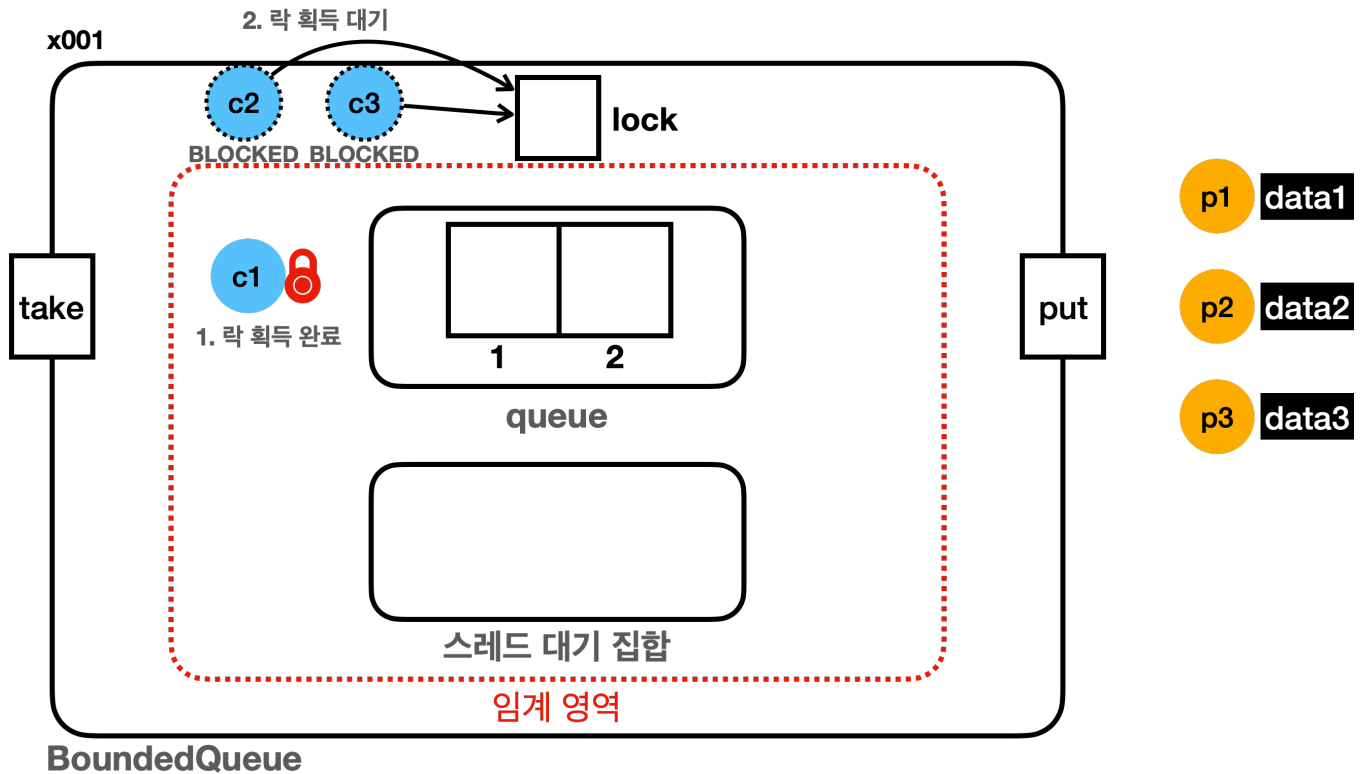
`synchronized` 대기

- 대기1: 락 획득 대기
 - `BLOCKED` 상태로 락 획득 대기
 - `synchronized`를 시작할 때 락이 없으면 대기
 - 다른 스레드가 `synchronized`를 빠져나갈 때 대기가 풀리며 락 획득 시도
- 대기2: `wait()` 대기
 - `WAITING` 상태로 대기
 - `wait()`를 호출 했을 때 스레드 대기 집합에서 대기
 - 다른 스레드가 `notify()`를 호출 했을 때 빠져나감

2가지 대기 상태를 그림으로 정리해보자.



- 소비자 스레드: `c1`, `c2`, `c3`
- 생산자 스레드: `p1`, `p2`, `p3`



- 소비자 스레드 **c1**, **c2**, **c3** 가 동시에 실행된다고 가정하자.
- 소비자 스레드 **c1** 이 가장 먼저 락을 획득한다.
- **c2**, **c3** 는 락 획득을 대기하며 **BLOCKED** 상태가 된다.

c2, **c3** 는 락 획득을 시도하지만, 모니터 락이 없기 때문에 락을 대기하며 **BLOCKED** 상태가 된다.

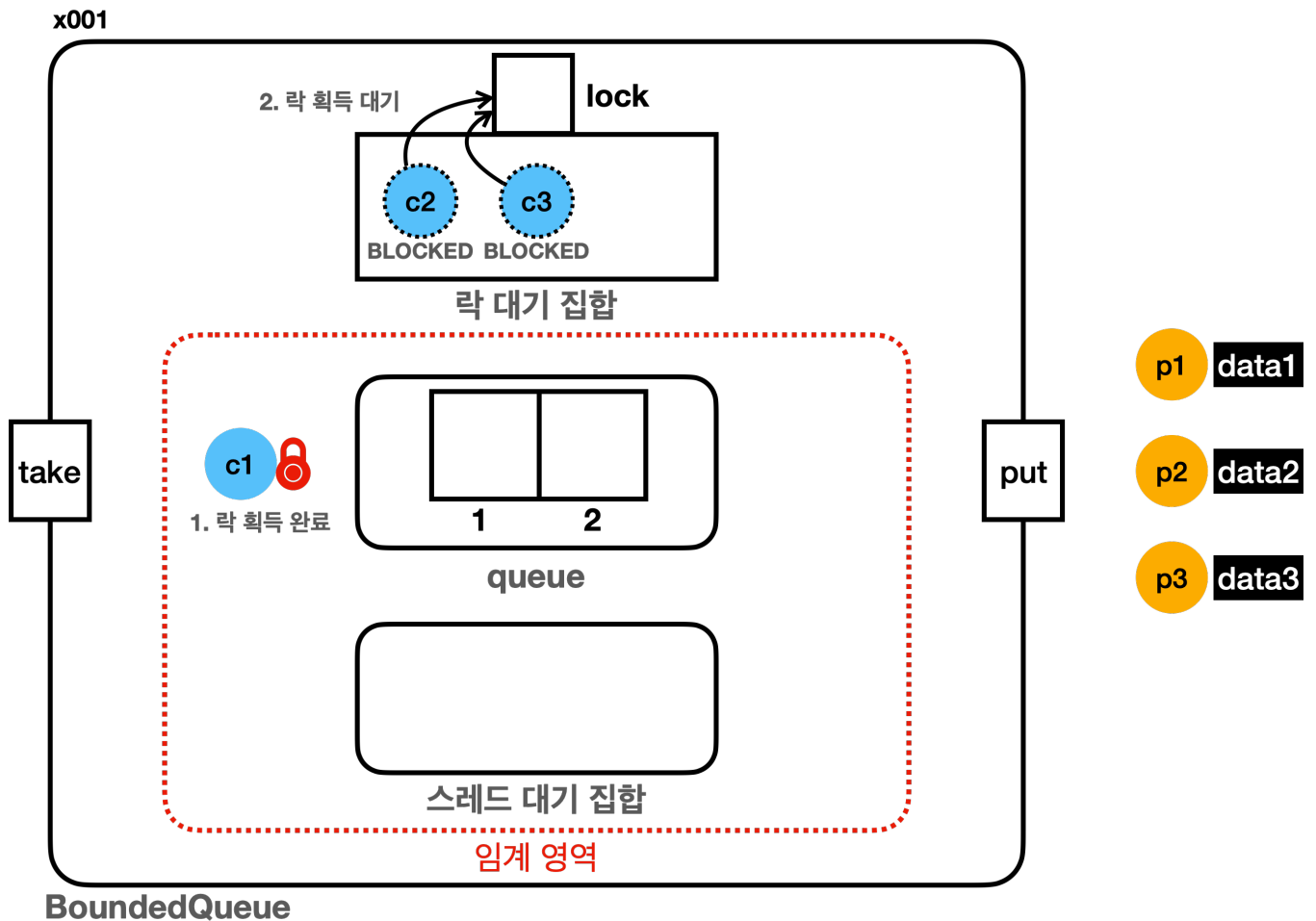
c1 은 나중에 락을 반납할 것이다. 그러면 **c2**, **c3** 중에 하나가 락을 획득해야 한다.

그런데 잘 생각해보면 락을 기다리는 **c2**, **c3** 도 어딘가에서 관리가 되어야 한다. 그래야 락이 반환되었을 때 자바가 **c2**, **c3** 중에 하나를 선택해서 락을 제공할 수 있다. 예를 들어서 **List**, **Set**, **Queue** 같은 자료구조에 관리가 되어야 한다.

그림에서는 **c2**, **c3** 가 단순히 **BLOCKED** 상태로 변경만 되었다. 그래서 관리되는 것 처럼 보이지는 않는다.

사실은 **BLOCKED** 상태의 스레드도 자바 내부에서 따로 관리된다. 다음 그림을 보자.

락 대기 집합

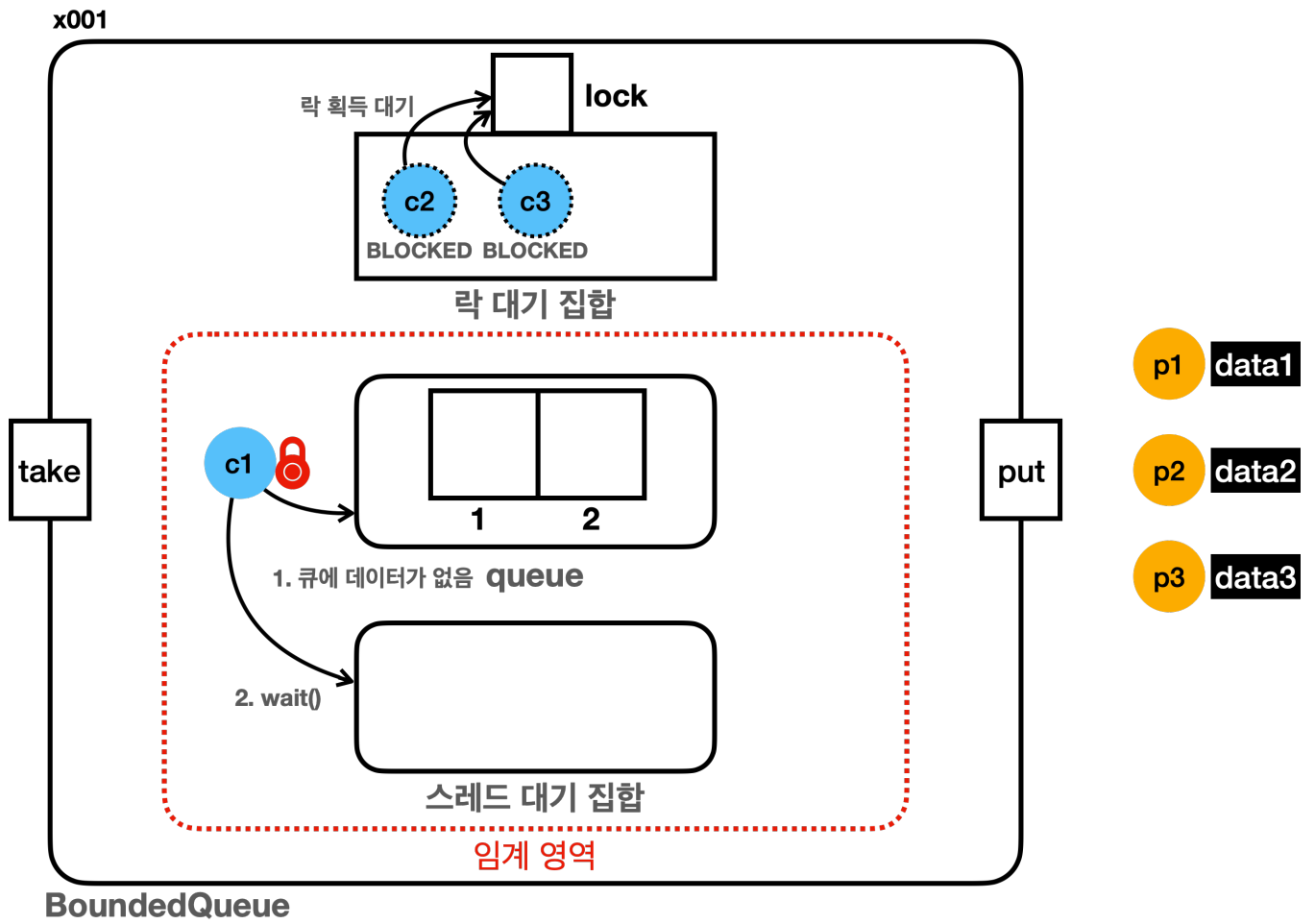


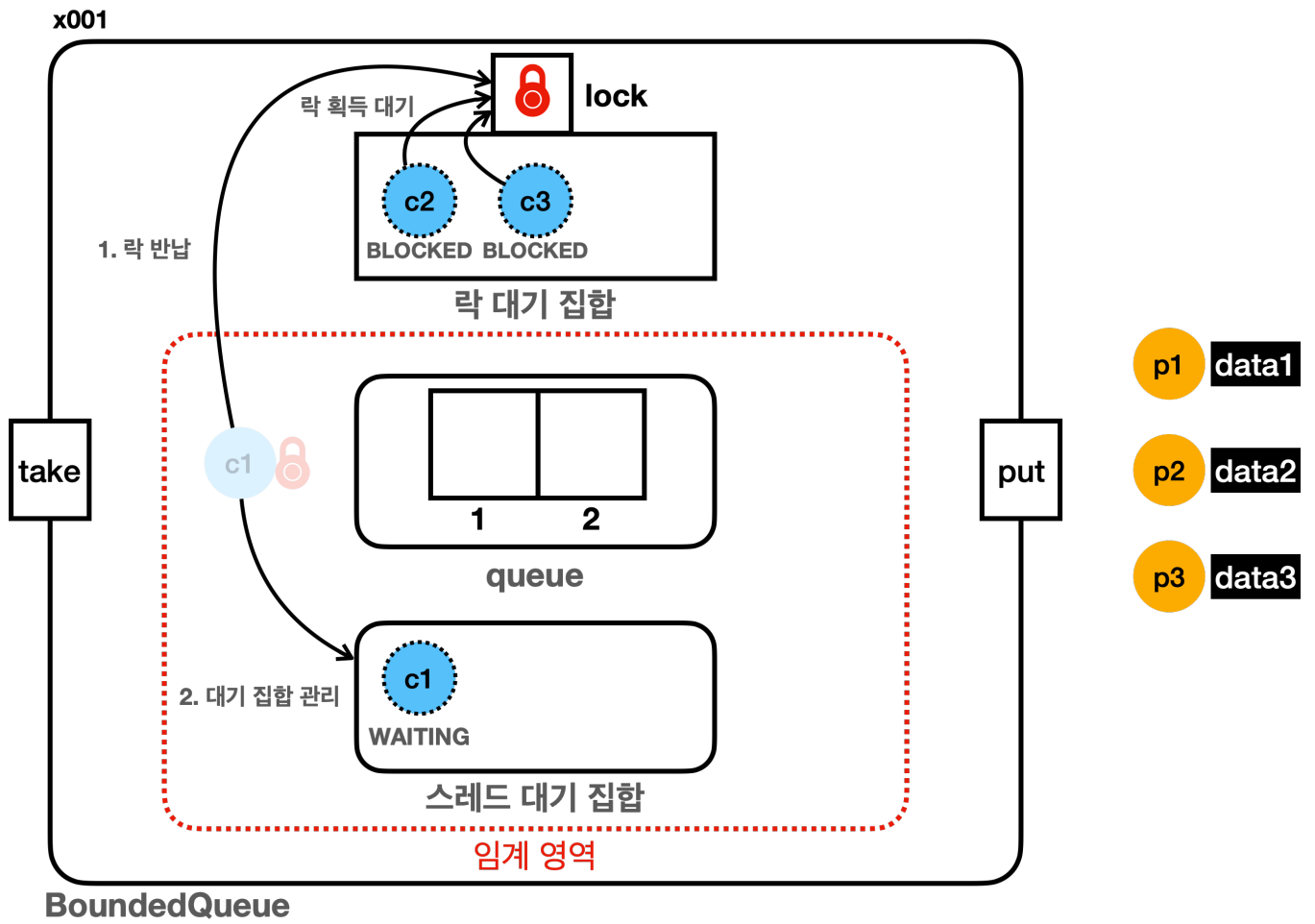
- 이 그림은 이전 그림과 같은 상태를 좀 더 자세히 그린 그림이다.
- 그림을 보면 락 대기 집합이라는 곳이 있다. 이곳은 락을 기다리는 **BLOCKED** 상태의 스레드들을 관리한다.
- 락 대기 집합은 자바 내부에 구현되어 있기 때문에 모니터 락과 같이 개발자가 확인하기는 어렵다.
- 여기서는 **BLOCKED** 상태의 스레드 **c2**, **c3** 가 관리된다.
- 언젠가 **c1** 이 락을 반납하면 락 대기 집합에서 관리되는 스레드 중 하나가 락을 획득한다.

락 대기 집합을 지금 설명하는 이유

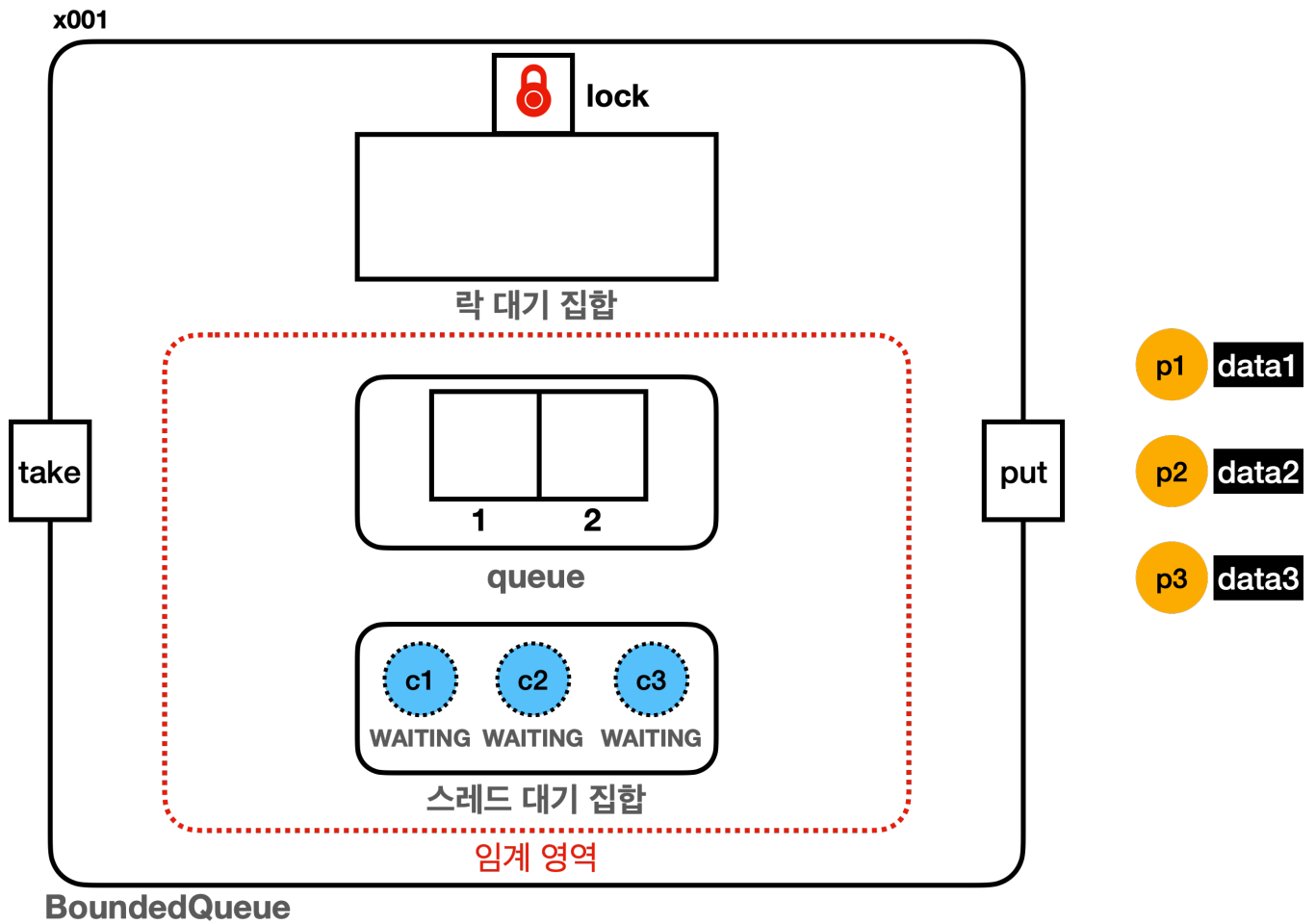
참고로 지금까지는 스레드를 최대한 쉽고 단순하게 설명하기 위해 **BLOCKED** 상태에서 사용하는 락 대기 집합을 일부러 설명하지 않았다. 이제 여러분이 스레드를 어느정도 이해했기 때문에 락 대기 집합의 개념을 설명해도 이해하는데 어려움은 없을 것이다.

사실 락 대기 집합에 대한 내용을 몰라도 괜찮다. 다만 지금 이 내용을 풀어서 설명하는 이유는 스레드가 모니터 락을 기다리는 상태와 **Object.wait()** 를 통한 대기 상태를 헷갈릴 수 있기 때문이다. 이 부분을 명확히 하기 위해 풀어서 설명한다.

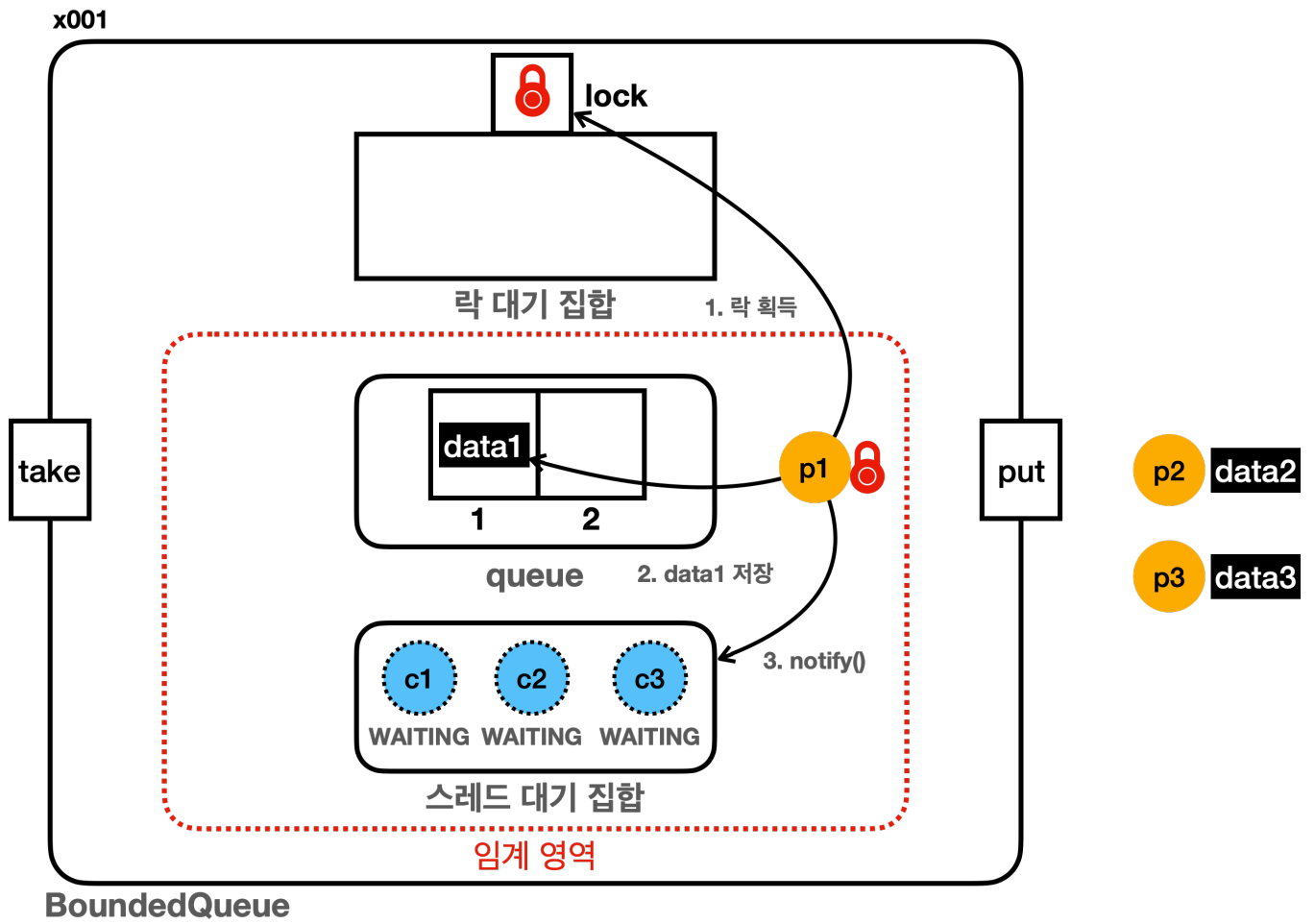




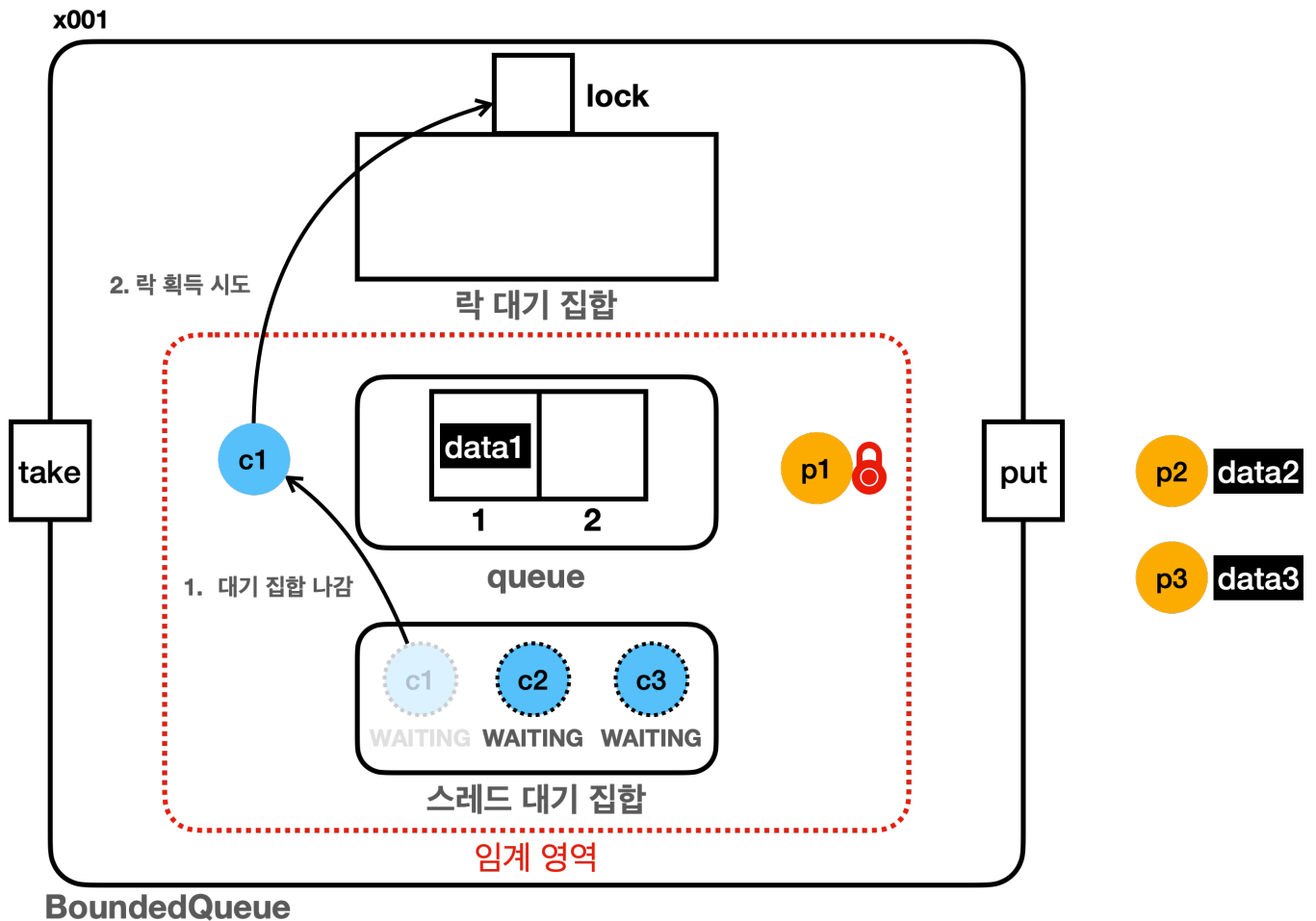
- **c1** 은 큐에 획득할 데이터가 없기 때문에 락을 반납하고, **WAITING** 상태로 스레드 대기 집합에서 대기한다.



- 이후에 락 대기 집합에 있는 **c2**가 락을 획득하고 임계 영역을 수행한다. 큐에 획득할 데이터가 없기 때문에 락을 반납하고, **WAITING** 상태로 스레드 대기 집합에서 대기한다.
- **c3**도 동일한 로직을 수행한다.



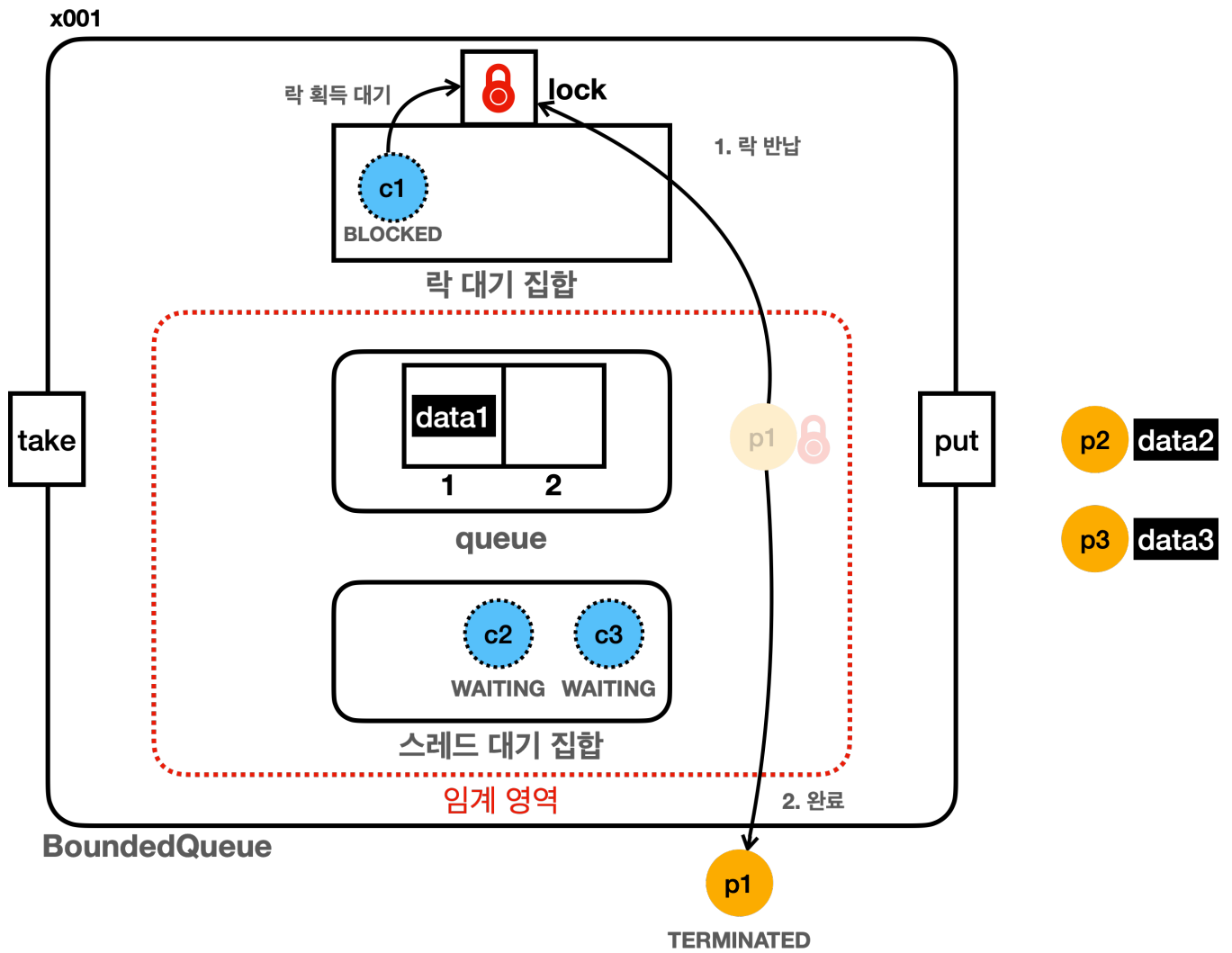
- p1 이 락을 획득하고 데이터를 저장한 다음 스레드 대기 집합에 이 사실을 알린다.



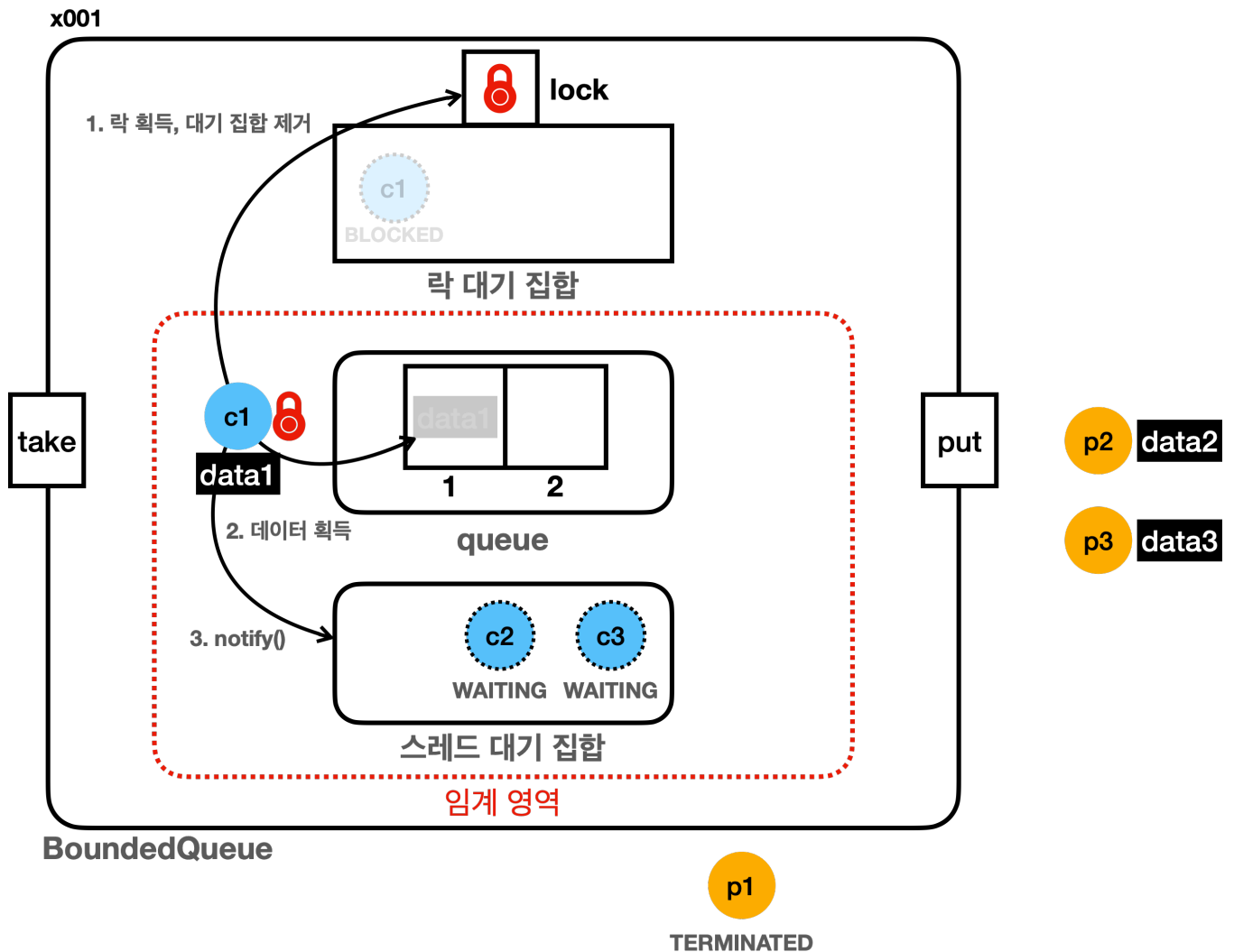
- 스레드 대기 집합에 있는 `c1` 이 스레드 대기 집합을 빠져나간다.
- 하지만 아직 끝난 것이 아니다. 락을 얻어서 락 대기 집합까지 빠져나가야 임계 영역을 수행할 수 있다.
- `c1` 은 락 획득을 시도하지만 락이 없다. 따라서 락 대기 집합에서 관리된다.

개념상 락 대기 집합이 1차 대기소이고, 스레드 대기 집합이 2차 대기소이다.

2차 대기소에 있는 스레드는 2차 대기소를 빠져나온다고 끝이 아니다. 1차 대기소까지 빠져나와야 임계 영역에서 로직을 수행할 수 있다. 비유를 하자면 임계 영역을 안전하게 지키기 위한 2중 감옥인 것이다. 스레드는 2중 감옥을 모두 탈출해야 임계 영역을 수행할 수 있다.



- **c1** 은 락 획득을 기다리며 **BLOCKED** 상태로 락 대기 집합에서 기다린다.
- 드디어 **p1** 이 락을 반납한다.



- 락이 반납되면 락 대기 집합에 있는 스레드 중 하나가 락을 획득한다. 여기서는 **c1** 이 락을 획득한다.
- **c1** 은 드디어 1차 대기소까지 탈출하고, 임계 영역을 수행한다.

정리

자바의 모든 객체 인스턴스는 멀티스레드와 임계 영역을 다루기 위해 내부에 3가지 기본 요소를 가진다.

- 모니터 락
- 락 대기 집합(모니터 락 대기 집합)
- 스레드 대기 집합

여기서 락 대기 집합이 1차 대기소이고, 스레드 대기 집합이 2차 대기소라 생각하면 된다. 2차 대기소에 들어간 스레드는 2차, 1차 대기소를 모두 빠져나와야 임계 영역을 수행할 수 있다.

이 3가지 요소는 서로 맞물려 돌아간다.

- **synchronized** 를 사용한 임계 영역에 들어가려면 모니터 락이 필요하다.
- 모니터 락이 없으면 락 대기 집합에 들어가서 **BLOCKED** 상태로 락을 기다린다.
- 모니터 락을 반납하면 락 대기 집합에 있는 스레드 중 하나가 락을 획득하고 **BLOCKED** -> **RUNNABLE** 상태가

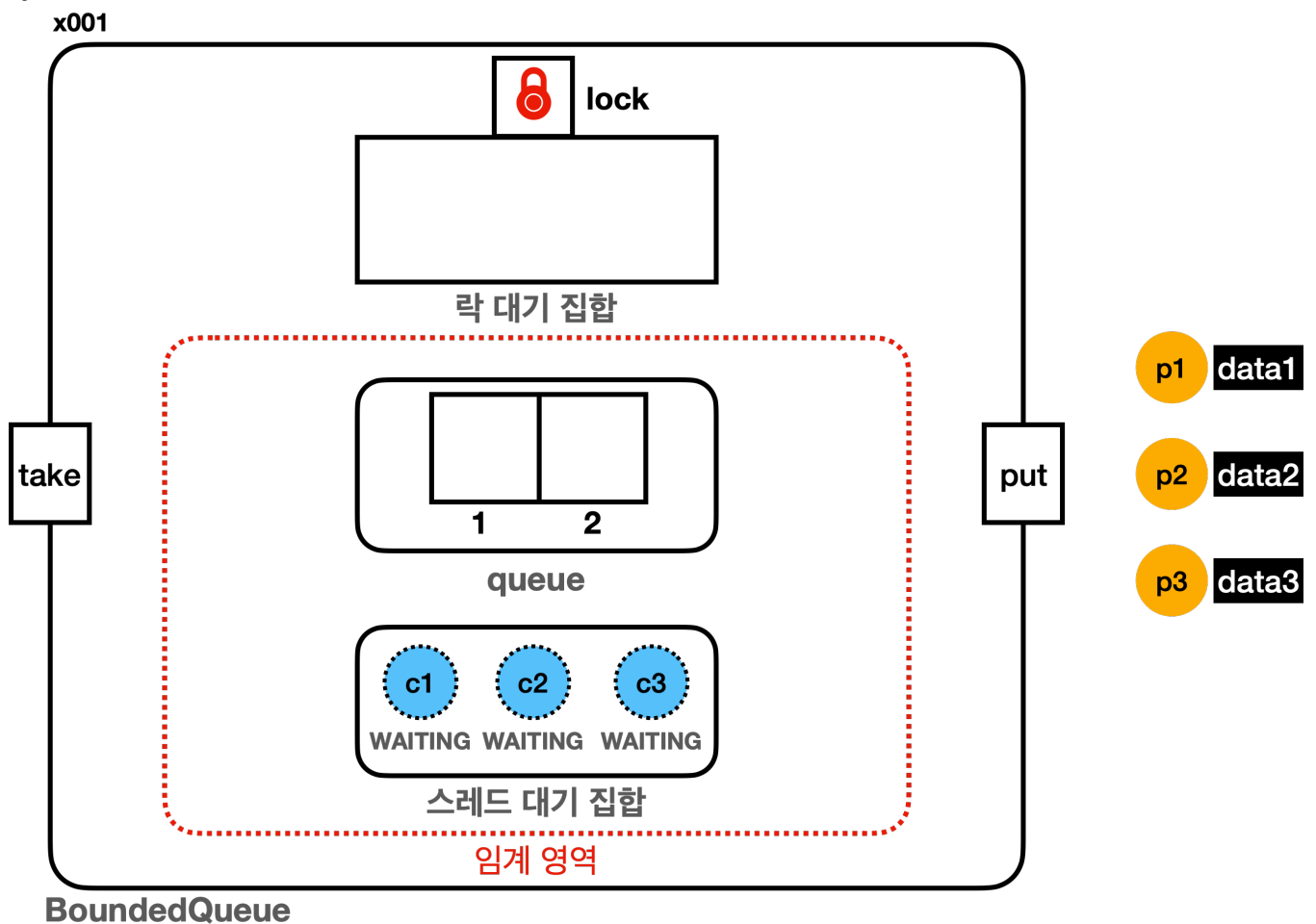
된다.

- `wait()` 를 호출해서 스레드 대기 집합에 들어가기 위해서는 모니터 락이 필요하다.
- 스레드 대기 집합에 들어가면 모니터 락을 반납한다.
- 스레드가 `notify()` 를 호출하면 스레드 대기 집합에 있는 스레드 중 하나가 스레드 대기 집합을 빠져나온다. 그리고 모니터 락 획득을 시도한다.
 - 모니터 락을 획득하면 임계 영역을 수행한다.
 - 모니터 락을 획득하지 못하면 락 대기 집합에 들어가서 `BLOCKED` 상태로 락을 기다린다.

synchronized vs ReentrantLock 대기

`synchronized` 와 마찬가지로 `Lock(ReentrantLock)` 도 2가지 단계의 대기 상태가 존재한다. 둘다 같은 개념을 구현한 것이기 때문에 비슷하다. 먼저 `synchronized` 대기를 정리해보자.

synchronized 대기

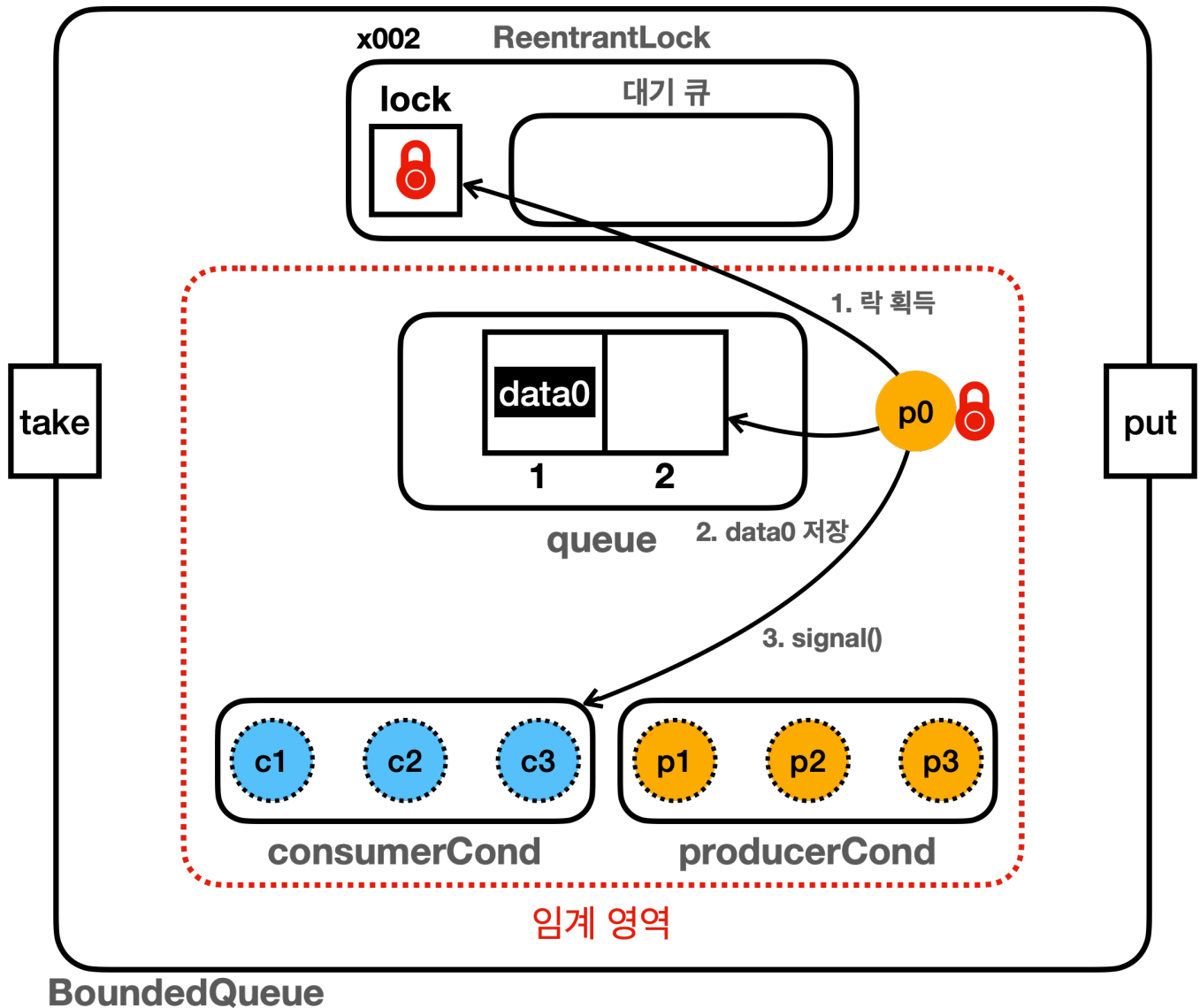


- 대기1: 모니터 락 획득 대기
 - 자바 객체 내부의 락 대기 집합(모니터 락 대기 집합)에서 관리
 - `BLOCKED` 상태로 락 획득 대기
 - `synchronized` 를 시작할 때 락이 없으면 대기

- 다른 스레드가 `synchronized` 를 빠져나갈 때 락을 획득 시도, 락을 획득하면 락 대기 집합을 빠져나감
- 대기2: `wait()` 대기
 - `wait()` 를 호출 했을 때 자바 객체 내부의 스레드 대기 집합에서 관리
 - `WAITING` 상태로 대기
 - 다른 스레드가 `notify()` 를 호출 했을 때 스레드 대기 집합을 빠져나감

Lock(ReentrantLock)

x001



참고: 영상과 그림이 다릅니다. 메뉴얼에 있는 그림이 맞습니다.

- 대기1: `ReentrantLock` 락 획득 대기
 - `ReentrantLock` 의 대기 큐에서 관리
 - `WAITING` 상태로 락 획득 대기
 - `lock.lock()` 을 호출 했을 때 락이 없으면 대기

- 다른 스레드가 `lock.unlock()` 을 호출 했을 때 대기가 풀리며 락 획득 시도, 락을 획득하면 대기 큐를 빠져나감
- 대기2: `await()` 대기
 - `condition.await()` 를 호출 했을 때, `condition` 객체의 스레드 대기 공간에서 관리
 - `WAITING` 상태로 대기
 - 다른 스레드가 `condition.signal()` 을 호출 했을 때 `condition` 객체의 스레드 대기 공간에서 빠져나감

2단계 대기소

참고로 깨어난 스레드는 바로 실행되는 것이 아니다. `synchronized` 와 마찬가지로 `ReentrantLock` 도 대기소가 2단계로 되어 있다. 2단계 대기소인 `condition` 객체의 스레드 대기 공간을 빠져나온다고 바로 실행되는 것이 아니다.

임계 영역 안에서는 항상 락이 있는 하나의 스레드만 실행될 수 있다. 여기서는 `ReentrantLock` 의 락을 획득해야 `Runnable` 상태가 되면서 그 다음 코드를 실행할 수 있다. 락을 획득하지 못하면 `WAITING` 상태로 락을 획득할 때 까지 `ReentrantLock` 의 대기 큐에서 대기한다.

중간 정리 - 생산자 소비자 문제

생산자 소비자 문제, 또는 한정된 버퍼 문제를 해결하려면 단순한 자료 구조를 넘어서, 스레드를 제어할 수 있는 특별한 자료 구조가 필요하다.

지금까지 만든 `BoundedQueue` 의 구현체들을 간단하게 정리해보자.

BoundedQueueV1

- 단순한 큐 자료 구조이다. 스레드를 제어할 수 없기 때문에, 버퍼가 가득 차거나, 버퍼에 데이터가 없는 한정된 버퍼 상황에서 문제가 발생한다.
- 버퍼가 가득 찬 경우: 생산자의 데이터를 버린다.
- 버퍼에 데이터가 없는 경우: 소비자는 데이터를 획득할 수 없다. (`null`)

BoundedQueueV2

- 앞서 발생한 문제를 해결하기 위해 반복문을 사용해서 스레드를 대기하는 방법을 적용했다. 하지만 `synchronized` 임계 영역 안에서 락을 들고 대기하기 때문에, 다른 스레드가 임계 영역에 접근할 수 없는 문제가 발생했다. 결과적으로 나머지 스레드는 모두 `BLOCKED` 상태가 되고, 자바 스레드 세상이 멈추는 심각한 문제가 발생했다.

BoundedQueueV3

- `synchronized`와 함께 사용할 수 있는 `wait()`, `notify()`, `notifyAll()`을 사용해서 문제를 해결했다. `wait()`를 사용하면 스레드가 대기할 때, 락을 반납하고 대기한다. 이후에 `notify()`를 호출하면 스레드가 깨어나면서 락 획득을 시도한다. 이때 락을 획득하면 `RUNNABLE` 상태가 되고, 락을 획득하지 못하면 락 획득을 대기하는 `BLOCKED` 상태가 된다.
- 이렇게 해서 스레드를 제어하는 큐 자료 구조를 만들 수 있었다. 생산자 스레드는 버퍼가 가득차면 버퍼에 여유가 생길 때 까지 대기한다. 소비자 스레드는 버퍼에 데이터가 없으면 버퍼에 데이터가 들어올 때 까지 대기한다.
- 이런 구현 덕분에 단순한 자료 구조를 넘어서 스레드까지 제어할 수 있는 자료 구조를 완성했다.
- 이 방식의 단점은 스레드가 대기하는 대기 집합이 하나이기 때문에, 원하는 스레드를 선택해서 깨울 수 없다는 문제가 있었다. 예를 들어서 생산자는 데이터를 생산한 다음 대기하는 소비자를 깨워야 하는데, 대기하는 생산자를 깨울 수 있다. 따라서 비효율이 발생한다. 물론 이렇게 해도 비효율이 있을 뿐 로직은 모두 정상 작동한다.

BoundedQueueV4

- `synchronized`와 `wait()`, `notify()`를 사용해서 구현하면 스레드 대기 집합이 하나라는 단점이 있다. 이 단점을 극복하려면 스레드 대기 집합을 생산자 전용과 소비자 전용으로 나누어야 한다. 이렇게 하려면 `Lock(ReentrantLock)`을 사용해야 한다.
- 여기서는 단순히 `synchronized`와 `wait()`, `notify()`를 사용해서 구현한 코드를 `Lock(ReentrantLock)`를 사용하도록 변경했다. 다음으로 넘어가기 위한 중간 단계의 코드이다.
- 결과는 기존 코드와 같다.

BoundedQueueV5

- `Lock(ReentrantLock)`는 `Condition`이라는 스레드 대기 공간을 제공한다. 이 스레드 대기 공간을 원하는 만큼 따로 만들 수 있다.
 - `productCond`: 생산자 스레드를 위한 전용 대기 공간
 - `consumerCond`: 소비자 스레드를 위한 전용 대기 공간
- 덕분에 생산자가 데이터를 생산하고 나면 `consumerCond.signal()` 메서드를 통해 소비자 전용 대기 공간에 이 사실을 알릴 수 있다. 반대로 소비자가 데이터를 소비하고 나면 `productCond.signal()`을 통해 생산자 전용 대기 공간에 이 사실을 알릴 수 있다.
- 이렇게 스레드 대기 공간을 나누어서 앞서 `synchronized`, `wait()`, `notify()`를 사용한 방식에서 발생한 비효율 문제를 깔끔하게 해결할 수 있었다.

우리가 함께 완성한 `BoundedQueueV5`는 생산자 소비자 문제, 또는 한정된 버퍼라고 알려진 문제를 매우 효율적으로 해결할 수 있는 자료 구조이다. 이 자료 구조는 단순한 큐의 기능을 넘어서 스레드를 효과적으로 제어하는 기능도 포함한다.

만약 멀티스레드 상황에서 생산자 소비자 문제가 나타난다면 우리가 만든 `BoundedQueueV5`를 사용하면 된다.

이것은 큐 자료 구조인데, 여기에 한정된 버퍼 문제를 해결하기 때문에 앞에 `Bounded`라는 이름을 붙였다.

우리가 만든 `BoundedQueueV5` 를 보면 느끼는 점이 있을 것인데, 이것을 내가 사용하는 다양한 프로젝트에 재사용하거나 또는 다른 개발자들이 사용할 수 있게 코드를 배포해도 되겠다는 생각이 들 것이다. 사실 이런 생각이 들면, 이미 어딘가에 다 만들어져 있다!

BlockingQueue

`BoundedQueue` 를 스레드 관점에서 보면 큐가 특정 조건이 만족될 때까지 스레드의 작업을 차단(blocking)한다.

- **데이터 추가 차단:** 큐가 가득 차면 데이터 추가 작업(`put()`)을 시도하는 스레드는 공간이 생길 때까지 차단된다.
- **데이터 획득 차단:** 큐가 비어 있으면 획득 작업(`take()`)을 시도하는 스레드는 큐에 데이터가 들어올 때까지 차단된다.

그래서 스레드 관점에서 이 큐에 이름을 지어보면 `BlockingQueue` 라는 이름이 적절하다.

자바는 생산자 소비자 문제, 또는 한정된 버퍼라고 불리는 문제를 해결하기 위해

`java.util.concurrent.BlockingQueue` 라는 인터페이스와 구현체들을 제공한다.

그럼 처음부터 `BlockingQueue` 를 사용하면 되는데, 왜 이렇게 직접 구현했을까?

사실 생산자 소비자 문제는 멀티스레드의 기본기를 배울 수 있는 가장 좋은 예시이다.

왜 다른 스레드가 `BLOCKED` 상태에서 깨어날 수 없는지, `synchronized`, `Object.wait()`, `Object.notify()` 가 왜 필요한지, 한계점은 무엇인지, `ReentrantLock` 을 왜 만들었고, 또 `Condition` 은 왜 필요한지, 생산자 소비자를 왜 분리해야 하는지 등등, 이런 다양한 문제를 코드로 만들어가며 해결하는 과정을 통해 자연스럽게 멀티스레드의 기본기를 학습한 것이다.

이렇게 복잡한 생산자 소비자 문제를 직접 구현하며, 멀티스레드의 기본기를 잘 쌓아둔 덕분에 실무에서 복잡한 멀티스레드 상황을 만나도 잘 해쳐나갈 수 있을 것이다.

BlockingQueue - 예제6

자바는 생산자 소비자 문제를 해결하기 위해 `java.util.concurrent.BlockingQueue` 라는 특별한 멀티스레드 자료 구조를 제공한다. 이것은 이름 그대로 스레드를 차단(Blocking) 할 수 있는 큐다.

- **데이터 추가 차단:** 큐가 가득 차면 데이터 추가 작업(`put()`)을 시도하는 스레드는 공간이 생길 때까지 차단된다.
- **데이터 획득 차단:** 큐가 비어 있으면 획득 작업(`take()`)을 시도하는 스레드는 큐에 데이터가 들어올 때까지 차단된다.

`BlockingQueue`는 인터페이스이고, 다음과 같은 다양한 기능을 제공한다.

`java.util.concurrent.BlockingQueue`

```
package java.util.concurrent;

public interface BlockingQueue<E> extends Queue<E> {

    boolean add(E e);
    boolean offer(E e);
    void put(E e) throws InterruptedException;
    boolean offer(E e, long timeout, TimeUnit unit) throws
    InterruptedException;

    E take() throws InterruptedException;
    E poll(long timeout, TimeUnit unit) throws InterruptedException;
    boolean remove(Object o);

    //...
}
```

- 주요 메서드만 정리했다.
- 데이터 추가 메서드: `add()`, `offer()`, `put()`, `offer(타임아웃)`
- 데이터 획득 메서드: `take()`, `poll(타임아웃)`, `remove(..)`
- `Queue`를 상속 받는다. 큐를 상속 받았기 때문에 추가로 큐의 기능들도 사용할 수 있다.

데이터 추가, 데이터 획득에 다양한 종류의 메서드가 제공 되는 것을 확인할 수 있다. 이 부분은 조금 뒤에서 정리하며 설명하겠다.

`BlockingQueue` 인터페이스의 대표적인 구현체

- `ArrayBlockingQueue`: 배열 기반으로 구현되어 있고, 버퍼의 크기가 고정되어 있다.
- `LinkedBlockingQueue`: 링크 기반으로 구현되어 있고, 버퍼의 크기를 고정할 수도, 또는 무한하게 사용할 수도 있다.

참고: `Deque`용 동시성 자료 구조인 `BlockingDeque`도 있다. 동시성 자료 구조들은 뒤에서 다시 한번 설명한다.

이제 `BlockingQueue`를 사용하도록 기존 코드를 변경해보자.

```

package thread.bounded;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class BoundedQueueV6_1 implements BoundedQueue {

    private BlockingQueue<String> queue;

    public BoundedQueueV6_1(int max) {
        queue = new ArrayBlockingQueue<>(max);
    }

    public void put(String data) {
        try {
            queue.put(data);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    public String take() {
        try {
            return queue.take();
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public String toString() {
        return queue.toString();
    }
}

```

- `BlockingQueue.put(data)`: 앞서 설명한 `BoundedQueueV5.put()` 과 같은 기능을 제공한다.
- `BlockingQueue.take()`: 앞서 설명한 `BoundedQueueV5.take()` 와 같은 기능을 제공한다.

`ArrayBlockingQueue.put()` 의 코드를 확인해보자.

ArrayBlockingQueue.put()

```
public class ArrayBlockingQueue {

    final Object[] items;
    int count;
    ReentrantLock lock;
    Condition notEmpty; //소비자 스레드가 대기하는 condition
    Condition notFull; //생산자 스레드가 대기하는 condition

    public void put(E e) throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == items.length) {
                notFull.await();
            }
            enqueue(e);
        } finally {
            lock.unlock();
        }
    }

    private void enqueue(E e) {
        items[putIndex] = e;
        count++;
        notEmpty.signal();
    }
}
```

- 주요 코드만 가지고 왔다.
- 앞서 우리가 구현한 BoundedQueueV5와 비슷하게 구현되어 있다. ArrayBlockingQueue는 내부에서 ReentrantLock을 사용한다. 그리고 생산자 전용 대기실과 소비자 전용 대기실이 있다. 만약 버퍼가 가득 차면 생산자 스레드는 생산자 전용 대기실에서 대기(await())한다. 생산자 스레드가 생산을 완료하면 소비자 전용 대기실에 signal()로 신호를 전달한다.

우리가 구현한 기능과 차이가 있다면 인터럽트가 걸릴 수 있도록, lock.lock() 대신에 lock.lockInterruptibly()을 사용한 점과, 내부 자료 구조의 차이 정도이다. (참고로 lock.lock()은 인터럽트를 무시한다.)

BoundedMain - BoundedQueueV6_1을 사용하도록 변경하자

```
//BoundedQueue queue = new BoundedQueueV5(2);  
BoundedQueue queue = new BoundedQueueV6_1(2);
```

실행 결과

```
10:43:35.163 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV6_1 ==  
  
10:43:35.165 [      main] 생산자 시작  
10:43:35.170 [producer1] [생산 시도] data1 -> []  
10:43:35.170 [producer1] [생산 완료] data1 -> [data1]  
10:43:35.268 [producer2] [생산 시도] data2 -> [data1]  
10:43:35.268 [producer2] [생산 완료] data2 -> [data1, data2]  
10:43:35.373 [producer3] [생산 시도] data3 -> [data1, data2]  
  
10:43:35.474 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]  
10:43:35.475 [      main] producer1: TERMINATED  
10:43:35.475 [      main] producer2: TERMINATED  
10:43:35.475 [      main] producer3: WAITING  
  
10:43:35.475 [      main] 소비자 시작  
10:43:35.476 [consumer1] [소비 시도]      ? <- [data1, data2]  
10:43:35.476 [producer3] [생산 완료] data3 -> [data2, data3]  
10:43:35.476 [consumer1] [소비 완료] data1 <- [data2]  
10:43:35.581 [consumer2] [소비 시도]      ? <- [data2, data3]  
10:43:35.581 [consumer2] [소비 완료] data2 <- [data3]  
10:43:35.686 [consumer3] [소비 시도]      ? <- [data3]  
10:43:35.686 [consumer3] [소비 완료] data3 <- []  
  
10:43:35.791 [      main] 현재 상태 출력, 큐 데이터: []  
10:43:35.791 [      main] producer1: TERMINATED  
10:43:35.791 [      main] producer2: TERMINATED  
10:43:35.792 [      main] producer3: TERMINATED  
10:43:35.792 [      main] consumer1: TERMINATED  
10:43:35.792 [      main] consumer2: TERMINATED  
10:43:35.792 [      main] consumer3: TERMINATED  
10:43:35.792 [      main] == [생산자 먼저 실행] 종료, BoundedQueueV6_1 ==
```

- 실행 결과는 앞서 만든 BoundedQueueV5와 같기 때문에 생산자 먼저 실행만 출력했다.
- BlockingQueue의 구현체가 내부에서 모든 로그를 출력하지는 않기 때문에 로그의 양은 줄어들었다. 실제 기능은 BoundedQueueV5와 같다고 생각하면 된다.
- 결과를 보면 모든 소비자는 자료를 잘 소비했고, 큐에 데이터도 비어있는 것을 확인할 수 있다. 모든 스레드도 정

상 종료되었다.

BlockingQueue - 기능 설명

실무에서 멀티스레드를 사용할 때는 응답성이 중요하다. 예를 들어서 대기 상태에 있어도, 고객이 중지 요청을 하거나, 또는 너무 오래 대기한 경우 포기하고 빠져나갈 수 있는 방법이 필요하다.

생산자가 무언가 데이터를 생산하는데, 버퍼가 빠지지 않아서 너무 오래 대기해야 한다면, 무한정 기다리는 것 보다는 작업을 포기하고, 고객분께는 "죄송합니다. 현재 시스템에 문제가 있습니다. 나중에 다시 시도해주세요." 라고 하는 것이 더 나은 선택일 것이다.

예를 들어서 생산자는 서버에 상품을 주문하는 고객일 수 있다. 고객이 상품을 주문하면, 고객의 요청을 생산자 스레드가 받아서 중간에 있는 큐에 넣어준다고 가정하자. 소비자 스레드는 큐에서 주문 요청을 꺼내서 주문을 처리하는 스레드이다. 만약에 선착순 할인 이벤트가 크게 성공해서 갑자기 주문이 폭주하면 주문을 만드는 생산자 스레드는 매우 바쁘게 주문을 큐에 넣게 된다.

큐의 한계가 1000개라고 가정하자. 생산자 스레드는 순간적으로 1000개가 넘는 주문을 큐에 담았다. 소비자 스레드는 한 번에 겨우 10개 정도의 주문만 처리할 수 있다. 이 상황에서 생산자 스레드는 계속 생산을 시도한다. 결국 소비가 생산을 따라가지 못하고, 큐가 가득 차게 된다.

이런 상황이 되면 수 많은 생산자 스레드는 큐 앞에서 대기하게 된다. 결국 고객도 응답을 받지 못하고 무한 대기하게 된다. 고객 입장에서 무작정 무한 대기하고 결과도 알 수 없는 상황이 가장 나쁜 상황일 것이다.

이렇게 생산자 스레드가 큐에 데이터를 추가할 때 큐가 가득 찬 경우, 또는 큐에 데이터를 추가하기 위해 너무 오래 대기한 경우에는 데이터 추가를 포기하고, 고객에게 주문 폭주로 너무 많은 사용자가 몰려서 요청을 처리할 수 없다거나, 또는 나중에 다시 시도해달라고 하는 것이 더 나은 선택일 것이다.

큐가 가득 찼을 때 생각할 수 있는 선택지는 4가지가 있다.

- 예외를 던진다. 예외를 받아서 처리한다.
- 대기하지 않는다. 즉시 `false` 를 반환한다.
- 대기한다.
- 특정 시간 만큼만 대기한다.

이런 문제를 해결하기 위해 `BlockingQueue` 는 각 상황에 맞는 다양한 메서드를 제공한다.

BlockingQueue의 다양한 기능 - 공식 API 문서

Operation	Throws Exception	Special Value	Blocks	Times Out
-----------	------------------	---------------	--------	-----------

Insert(추가)	add(e)	offer(e)	put(e)	offer(e, time, ur
Remove(제거)	remove()	poll()	take()	poll(time, unit)
Examine(관찰)	element()	peek()	not applicable	not applicable

참고: 문서 프로그램의 버그로 마지막에 offer(e, time, unit) 부분이 잘려 보입니다.

Throws Exception - 대기시 예외

- **add(e)**: 지정된 요소를 큐에 추가하며, 큐가 가득 차면 `IllegalStateException` 예외를 던진다.
- **remove()**: 큐에서 요소를 제거하며 반환한다. 큐가 비어 있으면 `NoSuchElementException` 예외를 던진다.
- **element()**: 큐의 머리 요소를 반환하지만, 요소를 큐에서 제거하지 않는다. 큐가 비어 있으면 `NoSuchElementException` 예외를 던진다.

Special Value - 대기시 즉시 반환

- **offer(e)**: 지정된 요소를 큐에 추가하려고 시도하며, 큐가 가득 차면 `false` 를 반환한다.
- **poll()**: 큐에서 요소를 제거하고 반환한다. 큐가 비어 있으면 `null` 을 반환한다.
- **peek()**: 큐의 머리 요소를 반환하지만, 요소를 큐에서 제거하지 않는다. 큐가 비어 있으면 `null` 을 반환한다.

Blocks - 대기

- **put(e)**: 지정된 요소를 큐에 추가할 때까지 대기한다. 큐가 가득 차면 공간이 생길 때까지 대기한다.
- **take()**: 큐에서 요소를 제거하고 반환한다. 큐가 비어 있으면 요소가 준비될 때까지 대기한다.
- **Examine (관찰)**: 해당 사항 없음.

Times Out - 시간 대기

- **offer(e, time, unit)**: 지정된 요소를 큐에 추가하려고 시도하며, 지정된 시간 동안 큐가 비워지기를 기다리다가 시간이 초과되면 `false` 를 반환한다.
- **poll(time, unit)**: 큐에서 요소를 제거하고 반환한다. 큐에 요소가 없다면 지정된 시간 동안 요소가 준비되기를 기다리다가 시간이 초과되면 `null` 을 반환한다.
- **Examine (관찰)**: 해당 사항 없음.

참고로 `BlockingQueue` 의 모든 대기, 시간 대기 메서드는 인터럽트를 제공한다.

대기하는 `put(e)`, `take()` 는 앞의 예제에서 살펴보았다. 나머지도 하나씩 코드로 확인해보자.

BlockingQueue - 기능 확인

BlockingQueue - 즉시 반환

BlockingQueue의 `offer(data)`, `poll()`를 사용해서 스레드를 대기하지 않고 즉시 반환해보자.

```
package thread.bounded;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

import static util.MyLogger.log;

public class BoundedQueueV6_2 implements BoundedQueue {

    private BlockingQueue<String> queue;

    public BoundedQueueV6_2(int max) {
        queue = new ArrayBlockingQueue<>(max);
    }

    public void put(String data) {
        boolean result = queue.offer(data);
        log("저장 시도 결과 = " + result);
    }

    public String take() {
        return queue.poll();
    }

    @Override
    public String toString() {
        return queue.toString();
    }
}
```

두 메서드는 스레드가 대기하지 않는다.

- `offer(data)`는 성공하면 `true`를 반환하고, 버퍼가 가득 차면 즉시 `false`를 반환한다.
- `poll()` 버퍼에 데이터가 없으면 즉시 `null`을 반환한다.

BoundedMain - BoundedQueueV6_2를 사용하도록 변경하자

```
//BoundedQueue queue = new BoundedQueueV6_1(2);  
BoundedQueue queue = new BoundedQueueV6_2(2);
```

실행 결과

```
11:30:54.139 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV6_2 ==  
  
11:30:54.141 [      main] 생산자 시작  
11:30:54.145 [producer1] [생산 시도] data1 -> []  
11:30:54.146 [producer1] 저장 시도 결과 = true  
11:30:54.146 [producer1] [생산 완료] data1 -> [data1]  
11:30:54.245 [producer2] [생산 시도] data2 -> [data1]  
11:30:54.245 [producer2] 저장 시도 결과 = true  
11:30:54.245 [producer2] [생산 완료] data2 -> [data1, data2]  
11:30:54.350 [producer3] [생산 시도] data3 -> [data1, data2]  
11:30:54.350 [producer3] 저장 시도 결과 = false  
11:30:54.350 [producer3] [생산 완료] data3 -> [data1, data2]  
  
11:30:54.455 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]  
11:30:54.456 [      main] producer1: TERMINATED  
11:30:54.456 [      main] producer2: TERMINATED  
11:30:54.456 [      main] producer3: TERMINATED  
  
11:30:54.456 [      main] 소비자 시작  
11:30:54.457 [consumer1] [소비 시도]      ? <- [data1, data2]  
11:30:54.457 [consumer1] [소비 완료] data1 <- [data2]  
11:30:54.562 [consumer2] [소비 시도]      ? <- [data2]  
11:30:54.562 [consumer2] [소비 완료] data2 <- []  
11:30:54.667 [consumer3] [소비 시도]      ? <- []  
11:30:54.667 [consumer3] [소비 완료] null <- []  
  
11:30:54.772 [      main] 현재 상태 출력, 큐 데이터: []  
11:30:54.772 [      main] producer1: TERMINATED  
11:30:54.773 [      main] producer2: TERMINATED  
11:30:54.773 [      main] producer3: TERMINATED  
11:30:54.773 [      main] consumer1: TERMINATED  
11:30:54.773 [      main] consumer2: TERMINATED  
11:30:54.773 [      main] consumer3: TERMINATED  
11:30:54.774 [      main] == [생산자 먼저 실행] 종료, BoundedQueueV6_2 ==
```

- 생산자 먼저 실행만 돌려보자.
- 실행 결과를 보면 우리가 처음 작성한 `BoundedQueueV1` 과 비슷하다.

```
11:30:54.350 [producer3] [생산 시도] data3 -> [data1, data2]
11:30:54.350 [producer3] 저장 시도 결과 = false
```

- 버퍼가 가득 차있는 경우 데이터를 추가하지 않고 즉시 `false` 를 반환한다.

```
11:30:54.667 [consumer3] [소비 시도]      ? <- []
11:30:54.667 [consumer3] [소비 완료] null <- []
```

- 버퍼에 데이터가 없는 경우 대기하지 않고 `null` 을 반환한다.

BlockingQueue - 시간 대기

`BlockingQueue` 의 `offer(data, 시간)`, `poll(시간)` 를 사용해서, 특정 시간 만큼만 대기하도록 해보자.

```
package thread.bounded;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.TimeUnit;

import static util.MyLogger.log;

public class BoundedQueueV6_3 implements BoundedQueue {

    private BlockingQueue<String> queue;

    public BoundedQueueV6_3(int max) {
        queue = new ArrayBlockingQueue<>(max);
    }

    public void put(String data) {
        try {
            // 대기 시간 설정 가능
            boolean result = queue.offer(data, 1, TimeUnit.NANOSECONDS);
            log("저장 시도 결과 = " + result);
        }
    }
}
```

```

        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    public String take() {
        try {
            // 대기 시간 설정 가능
            return queue.poll(2, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public String toString() {
        return queue.toString();
    }
}

```

- `offer(data, 시간)` 는 성공하면 `true` 를 반환하고, 버퍼가 가득 차서 스레드가 대기해야 하는 상황이면, 지정한 시간까지 대기한다. 대기 시간을 지나면 `false` 를 반환한다.
 - 여기서는 확인을 목적으로 1 나노초(`NANOSECONDS`)로 설정했다.
- `poll(시간)` 버퍼에 데이터가 없어서 스레드가 대기해야 하는 상황이면, 지정한 시간까지 대기한다. 대기 시간을 지나면 `null` 을 반환한다.
 - 여기서는 2초(`SECONDS`)로 설정했다.

BoundedMain - BoundedQueueV6_3을 사용하도록 변경하자

```

//BoundedQueue queue = new BoundedQueueV6_2(2);
BoundedQueue queue = new BoundedQueueV6_3(2);

```

실행 결과

```

11:42:12.231 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV6_3 ==

11:42:12.232 [      main] 생산자 시작
11:42:12.237 [producer1] [생산 시도] data1 -> []
11:42:12.237 [producer1] 저장 시도 결과 = true

```

```

11:42:12.237 [producer1] [생산 완료] data1 -> [data1]
11:42:12.340 [producer2] [생산 시도] data2 -> [data1]
11:42:12.340 [producer2] 저장 시도 결과 = true
11:42:12.340 [producer2] [생산 완료] data2 -> [data1, data2]
11:42:12.440 [producer3] [생산 시도] data3 -> [data1, data2]
11:42:12.441 [producer3] 저장 시도 결과 = false
11:42:12.441 [producer3] [생산 완료] data3 -> [data1, data2]

11:42:12.546 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]
11:42:12.546 [      main] producer1: TERMINATED
11:42:12.546 [      main] producer2: TERMINATED
11:42:12.546 [      main] producer3: TERMINATED

11:42:12.546 [      main] 소비자 시작
11:42:12.547 [consumer1] [소비 시도]      ? <- [data1, data2]
11:42:12.547 [consumer1] [소비 완료] data1 <- [data2]
11:42:12.651 [consumer2] [소비 시도]      ? <- [data2]
11:42:12.651 [consumer2] [소비 완료] data2 <- []
11:42:12.756 [consumer3] [소비 시도]      ? <- []

11:42:12.861 [      main] 현재 상태 출력, 큐 데이터: []
11:42:12.862 [      main] producer1: TERMINATED
11:42:12.862 [      main] producer2: TERMINATED
11:42:12.863 [      main] producer3: TERMINATED
11:42:12.863 [      main] consumer1: TERMINATED
11:42:12.863 [      main] consumer2: TERMINATED
11:42:12.863 [      main] consumer3: TIMED_WAITING
11:42:12.864 [      main] == [생산자 먼저 실행] 종료, BoundedQueueV6_3 ==
11:42:14.761 [consumer3] [소비 완료] null <- []

```

- 생산자 먼저 실행만 돌려보자.

```

11:42:12.440 [producer3] [생산 시도] data3 -> [data1, data2]
11:42:12.441 [producer3] 저장 시도 결과 = false

```

- 생산을 담당하는 `offer(data, 1나노초)` 메서드는 버퍼가 가득 찬 경우 1나노초 만큼 대기한 다음에 `false` 를 반환한다.
- 참고로 `false` 상황을 예시로 보여주기 위해 이렇게 짧은 시간을 선택했다.

```

11:42:12.756 [consumer3] [소비 시도]      ? <- []

```

```
...
//약 2초의 시간이 흐름
...
11:42:14.761 [consumer3] [소비 완료] null <- []
```

- 소비를 담당하는 poll(2초) 메서드는 버퍼가 빈 경우 2초 만큼 대기한 다음에 null 을 반환한다.
- 여기서 consumer3 은 빈 버퍼를 2초간 대기하다가 2초 후에 null 을 반환 받는다.

BlockingQueue - 예외

BlockingQueue 의 add(data), remove() 를 사용해서, 대기시 예외가 발생하도록 해보자.

```
package thread.bounded;

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class BoundedQueueV6_4 implements BoundedQueue {

    private BlockingQueue<String> queue;

    public BoundedQueueV6_4(int max) {
        queue = new ArrayBlockingQueue<>(max);
    }

    public void put(String data) {
        queue.add(data); // java.lang.IllegalStateException: Queue full
    }

    public String take() {
        return queue.remove(); // java.util.NoSuchElementException
    }

    @Override
    public String toString() {
        return queue.toString();
    }
}
```

- add(data) 는 성공하면 true 를 반환하고, 버퍼가 가득 차면 즉시 예외가 발생한다.

- `java.lang.IllegalStateException: Queue full`
- `remove()` 는 버퍼에 데이터가 없으면, 즉시 예외가 발생한다.
- `java.util.NoSuchElementException`

BoundedMain - BoundedQueueV6_4을 사용하도록 변경하자

```
//BoundedQueue queue = new BoundedQueueV6_3(2);
BoundedQueue queue = new BoundedQueueV6_4(2);
```

실행 결과

```
11:50:55.125 [      main] == [생산자 먼저 실행] 시작, BoundedQueueV6_4 ==

11:50:55.127 [      main] 생산자 시작
11:50:55.131 [producer1] [생산 시도] data1 -> []
11:50:55.131 [producer1] [생산 완료] data1 -> [data1]
11:50:55.234 [producer2] [생산 시도] data2 -> [data1]
11:50:55.234 [producer2] [생산 완료] data2 -> [data1, data2]
11:50:55.339 [producer3] [생산 시도] data3 -> [data1, data2]
Exception in thread "producer3" java.lang.IllegalStateException: Queue full
    at java.base/java.util.AbstractQueue.add(AbstractQueue.java:98)
    at java.base/
java.util.concurrent.ArrayBlockingQueue.add(ArrayBlockingQueue.java:329)
    at thread.bounded.BoundedQueueV6_4.put(BoundedQueueV6_4.java:18)
    at thread.bounded.ProducerTask.run(ProducerTask.java:17)
    at java.base/java.lang.Thread.run(Thread.java:1583)

11:50:55.441 [      main] 현재 상태 출력, 큐 데이터: [data1, data2]
11:50:55.441 [      main] producer1: TERMINATED
11:50:55.441 [      main] producer2: TERMINATED
11:50:55.441 [      main] producer3: TERMINATED

11:50:55.442 [      main] 소비자 시작
11:50:55.442 [consumer1] [소비 시도]      ? <- [data1, data2]
11:50:55.442 [consumer1] [소비 완료] data1 <- [data2]
11:50:55.547 [consumer2] [소비 시도]      ? <- [data2]
11:50:55.547 [consumer2] [소비 완료] data2 <- []
11:50:55.652 [consumer3] [소비 시도]      ? <- []
Exception in thread "consumer3" java.util.NoSuchElementException
    at java.base/java.util.AbstractQueue.remove(AbstractQueue.java:117)
    at thread.bounded.BoundedQueueV6_4.take(BoundedQueueV6_4.java:22)
```

```
at thread.bounded.ConsumerTask.run(ConsumerTask.java:15)
at java.base/java.lang.Thread.run(Thread.java:1583)
```

```
11:50:55.758 [    main] 현재 상태 출력, 큐 데이터: []
11:50:55.758 [    main] producer1: TERMINATED
11:50:55.758 [    main] producer2: TERMINATED
11:50:55.758 [    main] producer3: TERMINATED
11:50:55.759 [    main] consumer1: TERMINATED
11:50:55.759 [    main] consumer2: TERMINATED
11:50:55.759 [    main] consumer3: TERMINATED
11:50:55.759 [    main] == [생산자 먼저 실행] 종료, BoundedQueueV6_4 ==
```

- 생산자 먼저 실행만 돌려보자.

```
11:50:55.339 [producer3] [생산 시도] data3 -> [data1, data2]
Exception in thread "producer3" java.lang.IllegalStateException: Queue full
    at java.base/java.util.AbstractQueue.add(AbstractQueue.java:98)
    at java.base/
java.util.concurrent.ArrayBlockingQueue.add(ArrayBlockingQueue.java:329)
    at thread.bounded.BoundedQueueV6_4.put(BoundedQueueV6_4.java:18)
    at thread.bounded.ProducerTask.run(ProducerTask.java:17)
    at java.base/java.lang.Thread.run(Thread.java:1583)
```

- 생산을 담당하는 `add(data)` 메서드는 버퍼가 가득 찬 경우 `IllegalStateException`이 발생한다. 오류 메시지는 `Queue full`이다.

```
11:50:55.652 [consumer3] [소비 시도]      ? <- []
Exception in thread "consumer3" java.util.NoSuchElementException
    at java.base/java.util.AbstractQueue.remove(AbstractQueue.java:117)
    at thread.bounded.BoundedQueueV6_4.take(BoundedQueueV6_4.java:22)
    at thread.bounded.ConsumerTask.run(ConsumerTask.java:15)
    at java.base/java.lang.Thread.run(Thread.java:1583)
```

- 소비를 담당하는 `remove()` 메서드는 버퍼가 빈 경우 `NoSuchElementException`이 발생한다.

BoundedQueue 제거

이제 `BoundedQueue`는 단순히 위임만 하기 때문에, 앞서 우리가 만든 `BoundedQueue`를 제거하고 대신에 `BlockingQueue`를 직접 사용해도 된다. 대신에 `BoundedQueue`를 사용하는 모든 코드를 `BlockingQueue`를

사용하도록 변경해주어야 한다.

```
public static void main(String[] args) {  
    //1. BoundedQueue 선택  
    BlockingQueue<String> queue = new ArrayBlockingQueue<>(2);  
  
    //2. 생산자, 소비자 실행 순서 선택, 반드시 하나만 선택!  
    producerFirst(queue); //생산자 먼저 실행  
    //consumerFirst(queue); // 소비자 먼저 실행  
}
```

BoundedMain, ConsumerTask, ProducerTask 등의 코드를 변경해야 한다.

참고로 기존 코드를 유지하기 위해 변경하지는 않겠다.

정리

Doug Lea

자바 1.5에 추가된 `java.util.concurrent` 패키지가 제공하는 `Lock`, `ReentrantLock`, `Condition`, `BlockingQueue` 등을 보면 참 견고하게 잘 만들어진 라이브러리라는 생각이 들 것이다.

아주 흥미로운 점은 이 코드들을 열어보면 모두 코드 작성자에 "Doug Lea"라는 이름이 있는 것을 확인할 수 있다.

Doug Lea는 컴퓨터 과학 교수로 동시성 프로그래밍, 멀티스레딩, 병렬 컴퓨팅, 알고리즘 및 데이터 구조 등의 분야에서 많은 업적을 만들었다. 특히 자바 커뮤니티 프로세스(JCP)의 일원으로 활동하면서, JSR-166이라는 자바 `java.util.concurrent` 패키지의 주요 설계 및 구현을 주도했다. (참고로 혼자 만든 것은 아니다. 대표자의 이름이 들어간다.)

`java.util.concurrent` 패키지가 제공하는 동시성 라이브러리는 매우 견고하고, 높은 성능을 낼 수 있도록 최적화 되어 있다. 그리고 다양한 동시성 시나리오를 대응할 수 있고, 무엇보다 개발자가 쉽고 편리하게 복잡한 동시성 문제를 다룰 수 있게 해준다.

그는 동시성 프로그래밍의 복잡한 개념들을 실용적이고 효율적인 구현으로 변환하는 데 큰 역할을 했다.

결론적으로, Doug Lea의 `java.util.concurrent` 패키지에 대한 기여는 자바의 동시성 프로그래밍을 크게 발전시키고, 이는 현대 자바 프로그래밍의 핵심적인 부분이 되었다.

수 많은 자바 동시성 라이브러리들 뿐만 아니라, `Queue`, `Deque` 같은 자료 구조의 코드를 열어 보면 Doug Lea의 이

름을 발견할 수 있을 것이다.